Trabajo práctico 2: Detección de spam con aprendizaje supervisado y no supervisado

Alejandro Quesada Jose Herrera Manfred Azofeifa

24 de octubre de 2026

Índice

1. Introducción **3**
1.1. Protocolo común . . . 3

2. Aprendizaje supervisado: regresión logística **4**
2.1. Evaluación del modelo . . . 4
2.2. Entrenamiento . . . 4
2.3. Verificación con datos linealmente separables . . . 4
2.4. Extracción de características y clasificación . . . 7
2.5. Reducción de dimensionalidad con PCA . . . 9
2.6. Auditoría de la cuadrícula de entrenamiento . . . 14

3. Detección de anomalías: umbral de verosimilitud **14**
3.1. Densidades por histograma . . . 14
3.2. Evaluación en 30 corridas . . . 18
3.3. Auditoría de dimensión, intervalos y cuantil . . . 20

4. Clasificación supervisada: modelos grandes de lenguaje **21**
4.1. Clasificación sin ajuste . . . 21
4.2. Modelos adicionales con cuantización NF4 . . . 23
4.3. Ajuste fino eficiente . . . 26

5. Pruebas unitarias **29**
5.1. Evaluación del modelo . . . 30
5.2. Entrenamiento . . . 30
5.3. Otras pruebas del punto 1 . . . 31
5.4. Protocolo experimental y caché . . . 31
5.5. Detector de anomalías (punto 2.1) . . . 32
5.6. Ejecución portátil de los puntos 1 y 2 . . . 32
5.7. Modelos de lenguaje (puntos 3.1 y 3.2) . . . 32

6. Comparación de resultados **33**

7. Conclusiones **33**
7.1. Limitaciones . . . 33

Índice de figuras

1. Datos sintéticos de entrenamiento y frontera verdadera. 5
2. Proceso de entrenamiento sobre los datos sintéticos. 6
   a. Entropía cruzada media en cada época. 6
   b. Frontera $p = 0.5$ tras 0, 10, 50, 200 y 1000 épocas. 6
3. Probabilidad estimada por el modelo final y frontera aprendida sobre los 200 puntos de prueba. 7
4. F1 en la validación interna de la primera corrida. Las demás corridas realizan su propia calibración. 8
5. Resultados de la regresión logística con MPNet en las 30 corridas. 9
   a. Métricas de cada corrida y su media. 9
   b. Matriz de confusión promedio. 9
6. Varianza explicada por componente y acumulada en el PCA descriptivo de todos los mensajes. 10
7. Todos los mensajes sobre las dos primeras componentes del PCA descriptivo. 11
8. Plano de las dos primeras componentes y separación entre clases en cada una de las k componentes conservadas. 12
   a. 50 % de D ($k = 384$). 12
   b. 25 % de D ($k = 192$). 12
9. F1 de las 30 corridas con MPNet, con y sin PCA. 13
10. Brecha de verosimilitud entre ham y spam en la validación interna de la primera partición, con 384 componentes. Arriba, por componente; abajo, acumulada hasta k. 16
11. Densidades de los ham de entrenamiento en las dos componentes principales (semilla 2026). 17
12. Puntajes de verosimilitud de la prueba por clase y umbral (semilla 2026). 17
13. Región que el detector acepta como normal y mensajes de prueba (semilla 2026). . . 18
14. F1 del detector de anomalías y de la regresión logística en las mismas 30 particiones. 19
15. Tasa de aciertos del detector de anomalías y de la regresión logística en las mismas 30 particiones. 20
16. Variación entre muestras de las predicciones fijas de Qwen; no representa variación entre entrenamientos del modelo. 22
17. Confusión promedio de Qwen. La abundancia de ham marcado como spam limita la precisión a pesar de detectar la mayor parte del spam. 23
18. F1 de las predicciones fijas de cada configuración. Las cajas resumen muestras solapadas, no entrenamientos independientes. 25
19. Tasa de aciertos bajo el mismo muestreo. Se informa junto con F1 para reflejar el desbalance entre ham y spam. 26
20. Pérdida de entrenamiento por paso y de validación antes y después de la única época. La validación usa solo filas del entrenamiento externo. 28
21. Matrices de confusión de Qwen 14B NF4 sin ajuste (izquierda) y con LoRA (derecha) en la misma prueba reservada. 29

1. Introducción

Este informe documenta el Trabajo práctico 2 del curso de Principios de Inteligencia Artificial. El trabajo consiste en detectar mensajes spam del dataset SMSSpamCollection [2, 1], un conjunto público de mensajes SMS etiquetados como *ham* o *spam*, con tres enfoques implementados en PyTorch [9]:

1. clasificación supervisada con regresión logística sobre las características de un codificador, con y sin reducción de dimensionalidad por análisis de componentes principales;
2. detección de anomalías con densidades por histograma y un umbral de verosimilitud;
3. clasificación con modelos grandes de lenguaje.

La implementación completa vive en el notebook `tp2_detector_spam.ipynb`, organizado en celdas de definiciones, configuración, experimentos y análisis. Las pruebas externas cargan las mismas definiciones sin ejecutar descargas ni experimentos. Las métricas, figuras y tablas provienen de la ejecución completa de sus secciones. La línea base 3.1 ya se evaluó en GPU; 3.2 agrega dos modelos con cuantización NF4 y 3.3 ajusta Qwen 14B con LoRA en una partición fija, con F1 de 0.9708 en su prueba reservada frente a 0.7120 sin ajuste.

1.1. Protocolo común

- **Etiquetas.** *ham* $= 0$ y *spam* $= 1$; spam es la clase positiva del F1-score y la anomalía del detector.
- **Métricas.** Tasa de aciertos y F1-score.
- **Particiones.** 30 particiones estratificadas de 70 % de entrenamiento y 30 % de prueba, una por semilla.
- **Muestreo de los modelos de lenguaje.** 30 muestras estratificadas del 30 % del dataset.
- **Codificador.** `sentence-transformers/all-mpnet-base-v2`, con vectores de 768 dimensiones (Sección 2.4).
- **Hiperparámetros de la regresión logística.** Calibrados por F1 en una validación interna de cada corrida, tomada solo de su entrenamiento (Sección 2.4).
- **Dimensión óptima del PCA.** El equipo confirmó $D = 384$ el 24 de octubre de 2026 para la regresión logística, la mitad de las 768 características (Sección 2.5).
- **Detector de anomalías.** Entre los casos de 1.5, el detector de histogramas usa $D = 2$, elegida en una validación interna de los 30 entrenamientos; las densidades se construyen solo con ham y el umbral es el cuantil 5 % de un 20 % de ham reservado (Sección 3). Esta variante difiere de reutilizar $D = 384$, como pide la lectura literal del apartado 2.1; su aceptación debe validarse con el docente.
- Punto 3.1: `Qwen/Qwen2.5-3B-Instruct`, prompt sin ejemplos en inglés y decodificación voraz; las respuestas ambiguas cuentan como ham. Las 30 muestras estratificadas del 30 % usan las semillas de los puntos 1 y 2 (Sección 4). Para 3.2 se confirmaron Mistral-7B-Instruct-v0.3 y Qwen2.5-14B-Instruct, ambos con NF4 de 4 bits y cálculo bfloat16. Para 3.3 se confirmó ajustar Qwen 14B con LoRA sobre NF4 y un entrenamiento con prueba reservada (Sección 4.3).

2. Aprendizaje supervisado: regresión logística

2.1. Evaluación del modelo

La regresión logística modela la probabilidad de que un mensaje sea spam como

$$p(y = 1 \mid x) = \sigma(x^\top w + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

La función `forward` es una implementación propia en PyTorch [9], sin `torch.nn`. Calcula los logits $z = Xw + b$ de las n muestras con un único producto matriz-vector y les aplica la sigmoide. Para que la sigmoide no se desborde con logits grandes, se evalúa con $e^{-|z|} \in (0, 1]$:

$$\sigma(z) = \begin{cases} \dfrac{1}{1 + e^{-|z|}} & z \geq 0, \\ \dfrac{e^{-|z|}}{1 + e^{-|z|}} & z < 0. \end{cases}$$

Ninguna de las dos ramas produce infinitos, ni en el valor ni en el gradiente, mientras que la forma directa $1/(1 + e^{-z})$ calcula e^{1000} para $z = -1000$ y su gradiente con Autograd resulta indefinido. Para conservar también $\sigma'(0) = 1/4$, el exponente se implementa seleccionando $-z$ si $z \geq 0$ y z en caso contrario. Usar directamente `abs` daría una derivada nula en cero con Autograd. Una prueba adicional verifica los gradientes de pesos y sesgo en ese punto; el entrenamiento conserva su gradiente analítico propio.

2.2. Entrenamiento

La función `train_logistic_regression` minimiza la entropía cruzada binaria media

$$L(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right], \qquad p_i = \sigma(x_i^\top w + b),$$

con descenso del gradiente por lotes completos, también implementado sin optimizadores de PyTorch. En cada época se evalúa p con `forward` y se aplica el gradiente exacto de L:

$$w \leftarrow w - \frac{\alpha}{n} X^\top (p - y), \qquad b \leftarrow b - \frac{\alpha}{n} \sum_{i=1}^{n} (p_i - y_i).$$

Los pesos se inicializan con una normal de desviación 0.01 generada con la semilla de la corrida y el sesgo, en cero, de modo que la misma semilla reproduce el mismo modelo. El entrenamiento ejecuta un número fijo de épocas con una tasa de aprendizaje α fija; cada experimento elige sus propios valores. El historial de pérdida guarda la pérdida inicial y la de cada época, calculada como $\log(1 + e^{z_i}) - y_i z_i$, expresión equivalente a la entropía cruzada que no se indefine cuando p_i vale exactamente 0 o 1.

2.3. Verificación con datos linealmente separables

Antes de usar los mensajes, se verifica el regresor en un caso donde la respuesta correcta se conoce. La frontera verdadera es la recta

$$w^{*\top} x + b^* = 0.6\, x_1 + 0.8\, x_2 - 0.5 = 0.$$

Como $\|w^*\| = 1$, ${w^*}^\top x + b^*$ es la distancia con signo de x a la recta. Cada punto se coloca a una distancia $|d| = 1 + 0.5\,|z|$, con $z \sim \mathcal{N}(0,1)$, del lado de su clase (la clase 1 donde $d > 0$) y a una posición $\mathcal{N}(0, 2^2)$ a lo largo de la recta. Entre las dos clases queda una franja vacía de ancho 2, así que los datos son linealmente separables por construcción. Se generan 200 puntos de entrenamiento (100 por clase) y otros 200 de prueba con otra semilla (Figura 1).

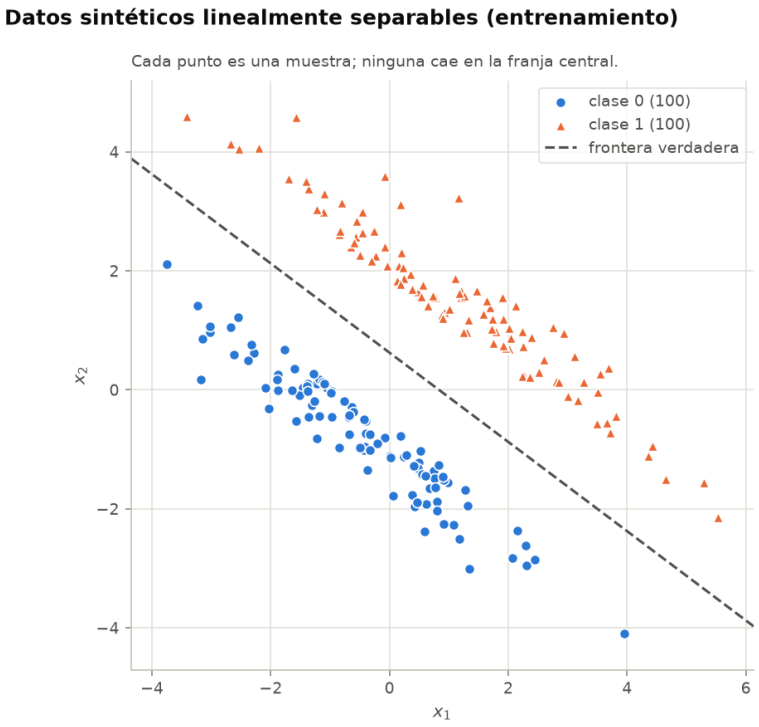
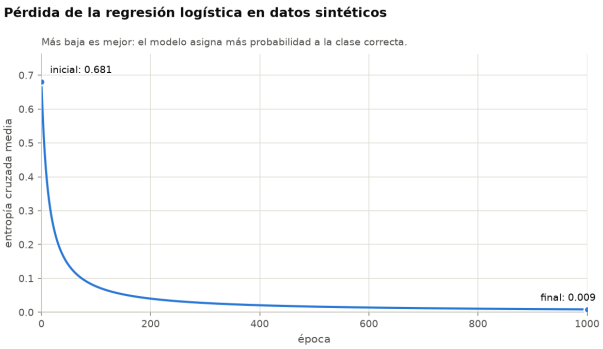
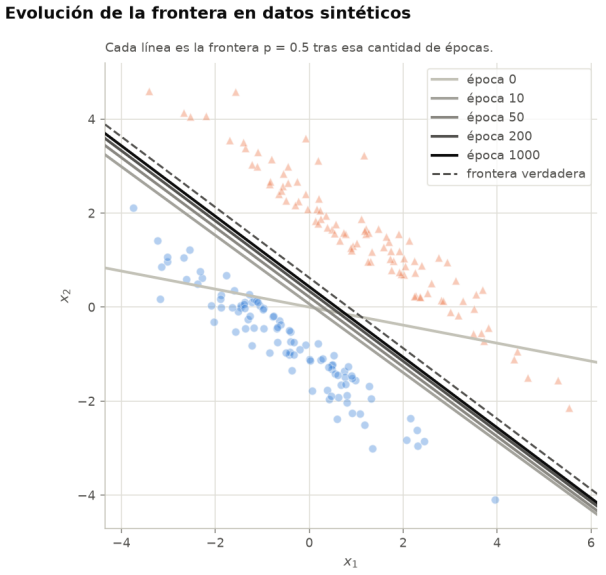


Figura 1: Datos sintéticos de entrenamiento y frontera verdadera.

El modelo se entrena con $\alpha = 0.1$ durante 1000 épocas. Como el entrenamiento es determinista para una semilla dada, entrenar k épocas reproduce exactamente el estado del entrenamiento completo en la época k; así se obtienen las fronteras intermedias de la Figura 2. La Tabla 1 compara cada estado con la frontera verdadera: como la recta $w^\top x + b = 0$ no cambia al multiplicar w y b por una constante, se reportan w y b divididos por $\|w\|$ y el ángulo entre w y w^*.



(a) Entropía cruzada media en cada época.



(b) Frontera $p = 0.5$ tras 0, 10, 50, 200 y 1000 épocas.

Figura 2: Proceso de entrenamiento sobre los datos sintéticos.

Tabla 1: Evolución del entrenamiento en datos sintéticos.

Época	Pérdida	Aciertos (entr.)	$w_1/\|w\|$	$w_2/\|w\|$	$b/\|w\|$	Ángulo con w^*
0	0.681	0.90	0.188	0.982	0.000	26.02°
10	0.388	1.00	0.590	0.808	−0.054	0.73°
50	0.139	1.00	0.596	0.803	−0.172	0.31°
200	0.040	1.00	0.598	0.801	−0.274	0.13°
1000	0.009	1.00	0.600	0.800	−0.347	0.01°
Verdadera	—	—	0.600	0.800	−0.500	0°

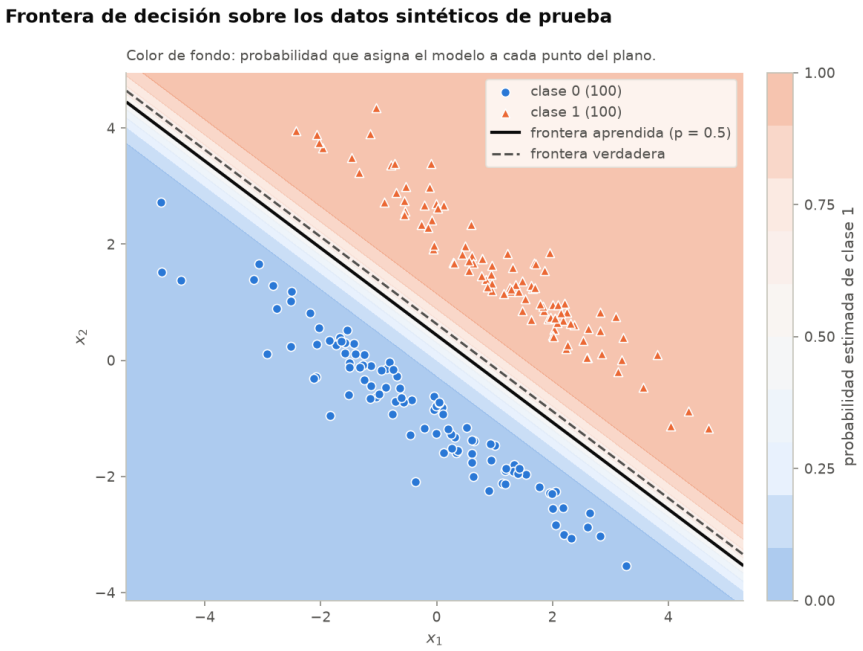


Figura 3: Probabilidad estimada por el modelo final y frontera aprendida sobre los 200 puntos de prueba.

El regresor aprende una frontera separadora casi paralela a la verdadera. En la época 0 los pesos son casi nulos ($\|w\| = 0.02$) y la frontera está girada 26° respecto de la verdadera; tras 10 épocas ya clasifica bien todo el entrenamiento y tras 1000 el ángulo es de 0.01°. La pérdida baja de 0.681 a 0.009 sin oscilar. En los 200 puntos de prueba la tasa de aciertos y el F1-score valen 1 (Figura 3).

Dos detalles muestran cómo aprende la regresión logística con datos separables. Primero, $\|w\|$ crece sin parar (de 0.02 a 3.84): como existe una recta que separa todo, la pérdida se puede seguir reduciendo haciendo más abrupta la transición de probabilidad, sin cambiar la recta; por eso la franja de colores alrededor de la frontera es angosta. Segundo, el sesgo normalizado (-0.35) todavía no alcanza el verdadero (-0.5): la frontera aprendida está desplazada 0.15 hacia la clase 0, pero dentro de la franja vacía, así que clasifica todo bien. Con datos separables hay infinitas rectas que separan perfectamente, y el descenso del gradiente se mueve muy lentamente hacia la de margen máximo [12]: la orientación converge mucho antes que la posición.

2.4. Extracción de características y clasificación

Codificador y datos. El archivo SMSSpamCollection de UCI contiene etiquetas y textos originales; sus columnas no son las dimensiones de entrada del clasificador. Cada mensaje se convierte en un embedding de $D = 768$ dimensiones con `all-mpnet-base-v2`, de sentence-transformers [11]. Se fija la revisión `e8c3b32edf5434bc2275fc9bab85f82640a19130` del modelo[1]. La ejecución produjo una matriz de forma $(5\,574, 768)$, en `float32`, con vectores de norma 1 y sin valores no finitos.

El codificador está preentrenado y no se ajusta con los mensajes de este dataset, por lo que las características se calculan una sola vez. La caché registra el identificador y la revisión del modelo, la dimensión y un hash SHA-256 de los textos ordenados. Antes de reutilizarla se comprueban esos metadatos, la forma y el tipo del tensor; una caché de MiniLM o de otros textos no se acepta. Los

[1] `https://huggingface.co/sentence-transformers/all-mpnet-base-v2`

datos sintéticos, el modelo y los hiperparámetros del apartado 1.3 no participan en este experimento ni en el apartado 1.5.

Calibración interna por corrida. Se realizan 30 particiones estratificadas, con semillas de 2026 a 2055. En cada partición, el 70 % se destina a entrenamiento y el 30 % a prueba; del entrenamiento se reserva un 20 % estratificado para validación interna y el resto ajusta los candidatos. La cuadrícula conserva todas las combinaciones de $\alpha \in \{0.1, 0.5, 1, 2, 5, 10, 20, 50\}$ y $\{250, 500, 1000, 2000, 4000\}$ épocas. Se selecciona automáticamente el mayor F1 en validación y, entre empates, la menor entropía cruzada de validación. Con la combinación elegida se vuelve a entrenar sobre todo el 70 % y solo entonces se evalúa el 30 % de prueba.

Cada corrida vuelve a calibrar: la selección de la primera partición no se reutiliza en las siguientes. De este modo, un mensaje reservado para prueba en una corrida no interviene en la calibración de esa misma corrida. La implementación entrena las tasas en paralelo y evalúa las épocas como checkpoints de sus trayectorias, con la misma inicialización para todos los candidatos; conserva las actualizaciones propias del descenso del gradiente. Una prueba contrasta la cuadrícula vectorizada con entrenamientos independientes.

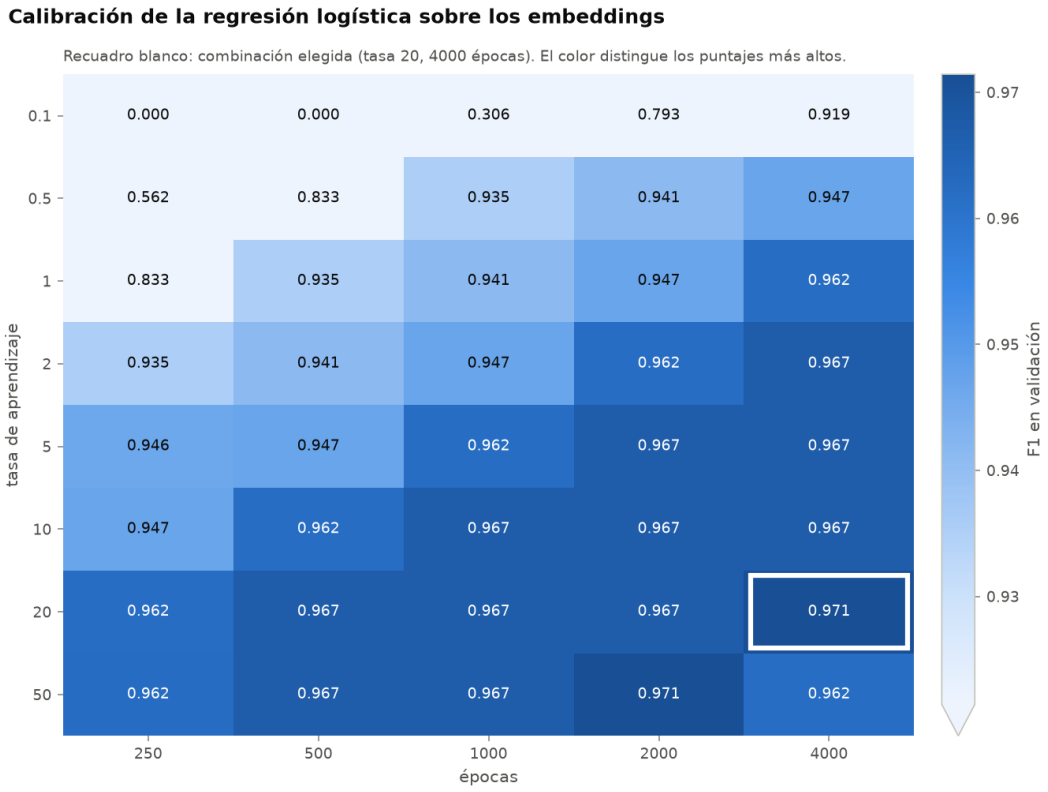


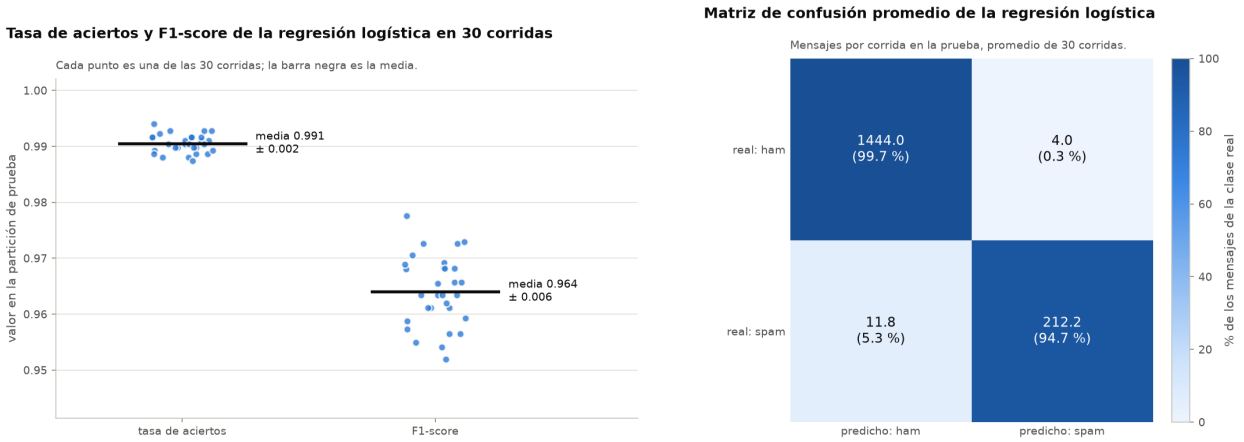
Figura 4: F1 en la validación interna de la primera corrida. Las demás corridas realizan su propia calibración.

En la primera corrida, la validación eligió $\alpha = 20$ y 4000 épocas, con F1 de 0.9714. Las 30 selecciones no fueron iguales: las tasas 10, 20 y 50 aparecieron en 6, 10 y 14 corridas, respectivamente; las épocas variaron entre 500 y 4000. En 15 corridas se seleccionó el máximo de 4000 épocas y en 14 la tasa máxima de 50, por lo que la cuadrícula no demuestra que se haya alcanzado un óptimo fuera de sus límites.

Resultados de las 30 corridas. La prueba de cada partición contiene 1 672 mensajes: 1 448 ham y 224 spam. Se conserva el umbral de probabilidad 0.5 y se registran la tasa de aciertos, el F1 de spam y los conteos de la matriz de confusión. La Tabla 2 resume las métricas y la Figura 5 muestra su dispersión y los errores medios.

Tabla 2: Regresión logística sobre MPNet de 768 dimensiones: 30 corridas con calibración interna en cada partición.

Métrica	Media	Desviación	Mínimo	Máximo
Tasa de aciertos	0.9906	0.0016	0.9874	0.9940
F1-score	0.9641	0.0063	0.9519	0.9776



(a) Métricas de cada corrida y su media.

(b) Matriz de confusión promedio.

Figura 5: Resultados de la regresión logística con MPNet en las 30 corridas.

La tasa de aciertos fue 0.9906 ± 0.0016 y el F1 de spam, 0.9641 ± 0.0063. El F1 varió entre 0.9519 y 0.9776. Las diferencias entre particiones muestran una dispersión acotada en este dataset, aunque no implican independencia estadística de las corridas. Como referencia, un clasificador que predijera siempre ham tendría una tasa de aciertos de $1448/1672 \simeq 0.8660$ y F1 de spam igual a cero. El resultado mejora ambas métricas y el F1 permite valorar la clase minoritaria. En promedio, se marcaron 4.0 ham como spam y se dejaron pasar 11.8 spam por corrida.

Límites de la evaluación. Las 30 particiones reutilizan mensajes del mismo dataset; su desviación describe la variación entre particiones y no es un intervalo de confianza de 30 muestras independientes. Además, la partición se hace por filas, sin agrupar los textos duplicados que existen en el archivo original. Por eso copias del mismo mensaje pueden aparecer en entrenamiento y prueba, con una posible sobreestimación del desempeño sobre mensajes nuevos. El dataset se conserva sin modificar; no se cuantifica el tamaño de ese posible sesgo con esta evaluación.

2.5. Reducción de dimensionalidad con PCA

Método. Con la matriz centrada $X_c = X - \mathbf{1}\mu^\top$ y su descomposición en valores singulares $X_c = U\Sigma V^\top$, las columnas de V son las componentes principales y la varianza de la componente j es

$\sigma_j^2/(n-1)$. La proyección conserva las k primeras: $Z = (X - \mathbf{1}\mu^\top)V_k$. La implementación calcula la SVD en doble precisión con `torch.linalg.svd` y fija el signo de cada componente haciendo positiva su coordenada de mayor valor absoluto. Esto elimina la ambigüedad de signo, aunque no garantiza identidad numérica entre todas las plataformas. Desde $D = 768$ se conservan 384 componentes (50 %), 192 (25 %) y 2.

Calibración y evaluación por dimensión. Cada valor de k y cada corrida tienen su propia selección de tasa y épocas, con la misma cuadrícula y las mismas particiones internas que el caso sin PCA. Durante la calibración, PCA se ajusta exclusivamente con la parte interna de ajuste, sin la validación. Elegidos los hiperparámetros, se ajusta de nuevo con todo el 70 % de entrenamiento y se aplica a la prueba restando esa misma media. Los distintos valores de k reutilizan las primeras componentes del mismo ajuste completo, sin mezclar índices de entrenamiento de otras particiones. Todas las variantes usan las mismas 30 particiones, por lo que las diferencias de F1 se comparan corrida a corrida.

Varianza y visualización. Para las figuras se ajusta un PCA sobre todos los mensajes, exclusivamente descriptivo: ese modelo no se usa para entrenar ni evaluar la clasificación. La Figura 6 muestra la varianza individual y acumulada.

Con 384 componentes se conserva el 96.16 % de la varianza; con 192, el 84.13 %, y con 2, el 8.08 %. Una reducción al 50 % de las coordenadas conserva casi toda la varianza, pero PCA ordena las direcciones por varianza y no por capacidad de separar ham de spam. Por eso esa proporción no sustituye la evaluación de clasificación.

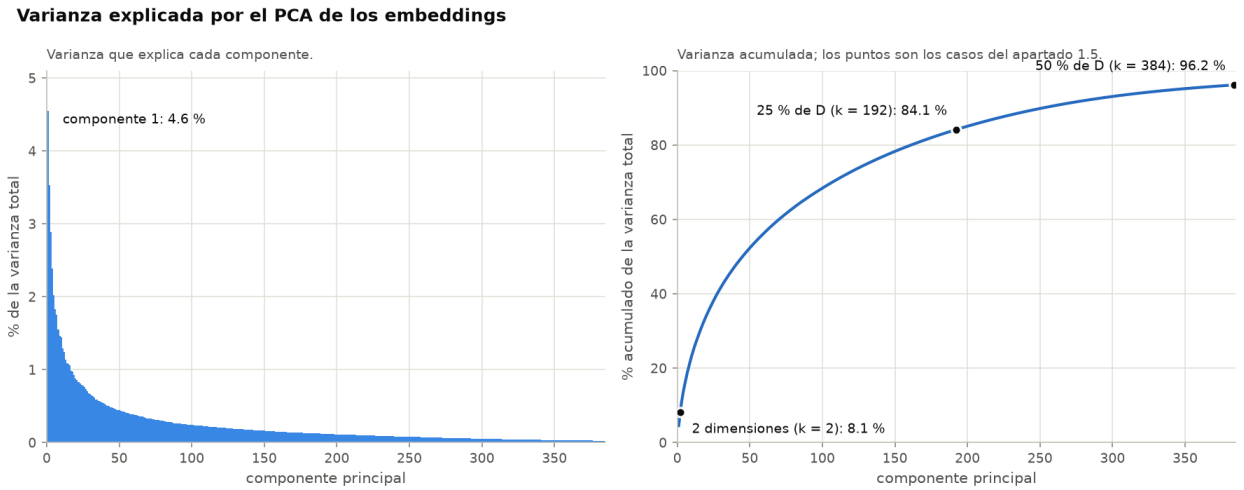


Figura 6: Varianza explicada por componente y acumulada en el PCA descriptivo de todos los mensajes.

Con 2 dimensiones se representan todos los mensajes en el plano (Figura 7). Para 192 y 384, el segundo panel muestra cuánto separan las clases las coordenadas de cada componente, usando

$$|d_j| = \frac{|\bar{z}_{j,\text{spam}} - \bar{z}_{j,\text{ham}}|}{\sqrt{(s^2_{j,\text{ham}} + s^2_{j,\text{spam}})/2}}.$$

Esta medida descriptiva compara la diferencia de medias con la dispersión de las clases y no se usa para elegir componentes o hiperparámetros.

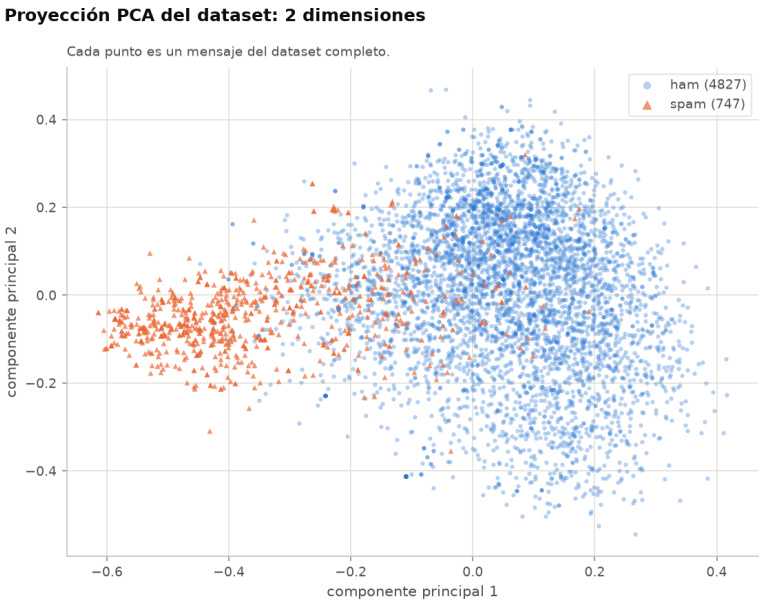
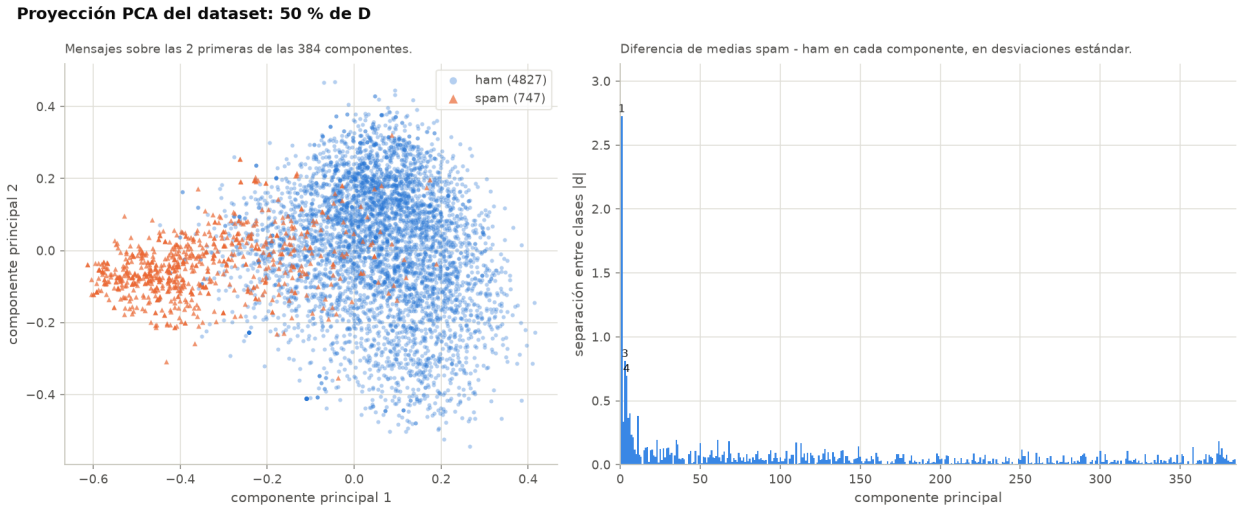
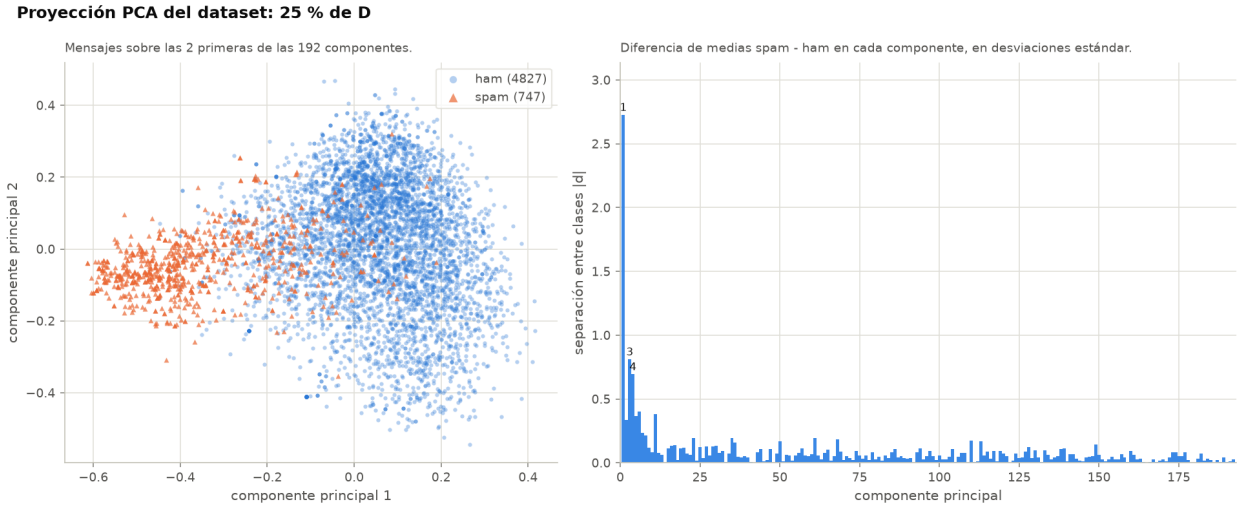


Figura 7: Todos los mensajes sobre las dos primeras componentes del PCA descriptivo.



(a) 50 % de D ($k = 384$).



(b) 25 % de D ($k = 192$).

Figura 8: Plano de las dos primeras componentes y separación entre clases en cada una de las k componentes conservadas.

Clasificación. La Tabla 3 compara las métricas y la diferencia pareada de F1 con respecto al caso sin PCA. La varianza corresponde al PCA descriptivo completo; las métricas proceden de los PCA ajustados de manera independiente dentro de cada corrida.

Tabla 3: Regresión logística con MPNet, con y sin PCA: media ± desviación en 30 corridas. ΔF1 compara cada variante con el caso sin PCA en la misma partición.

Dimensiones	Varianza	Tasa de aciertos	F1-score	ΔF1
768 (sin PCA)	100 %	0.9906 ± 0.0016	0.9641 ± 0.0063	—
384	96.16 %	0.9905 ± 0.0019	0.9637 ± 0.0075	-0.0004 ± 0.0030
192	84.13 %	0.9894 ± 0.0021	0.9596 ± 0.0082	-0.0045 ± 0.0054
2	8.08 %	0.9466 ± 0.0054	0.7845 ± 0.0189	-0.1796 ± 0.0198

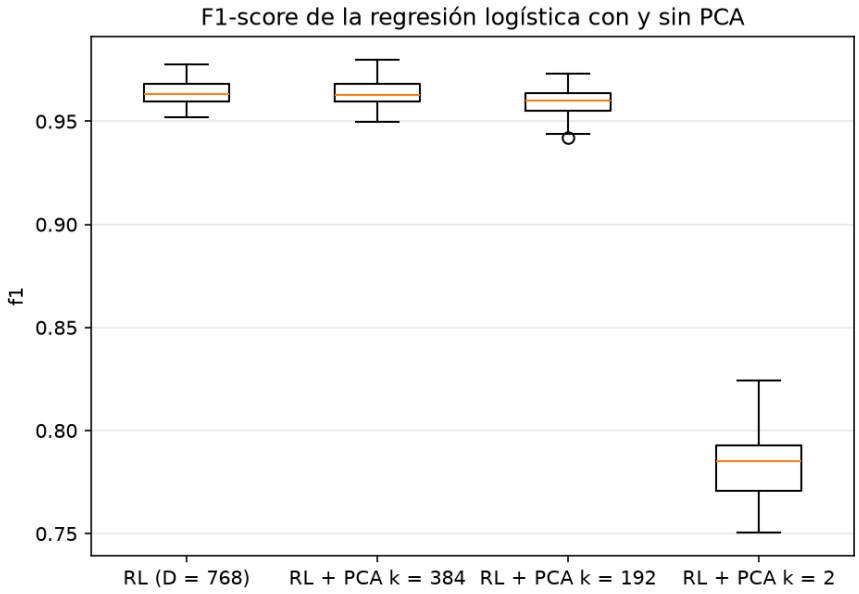


Figura 9: F1 de las 30 corridas con MPNet, con y sin PCA.

Con 384 componentes, el cambio medio de F1 fue de -0.0004, con una desviación pareada de 0.0030; el F1 fue menor que sin PCA en 14 de las 30 corridas. La diferencia media es pequeña y cambia de signo entre particiones, aunque esto no constituye una prueba formal de equivalencia. Con 192 componentes, el cambio fue de -0.0045 ± 0.0054, y el F1 bajó en 24 de las 30 corridas. Con 2 componentes, el cambio fue de -0.1796 ± 0.0198 y el F1 fue menor en las 30 corridas.

El tipo de error también cambia. Sin PCA se marcan en promedio 4.0 ham como spam y se dejan pasar 11.8 spam por corrida. Con 384 componentes, los promedios son 3.9 y 12.1; con 192, 4.3 y 13.4. Con 2 componentes suben a 27.9 y 61.4: se deja pasar alrededor del 27.4 % de los 224 spam de cada prueba. Su tasa de aciertos de 0.9466 esconde una pérdida mayor en la clase minoritaria, que el F1 de 0.7845 hace visible.

Las variantes recibieron calibración interna propia, por lo que se comparan después de buscar sus hiperparámetros en la misma cuadrícula. Esto reduce la dependencia de una configuración única, pero no permite atribuir toda diferencia exclusivamente a información descartada: el resultado también depende de una validación finita y de una búsqueda limitada. En particular, la tasa máxima 50 se eligió en 15 de las corridas con $k = 384$ y en 19 con $k = 192$.

D **óptima.** El equipo confirmó `OPTIMAL_PCA_DIMENSIONS = 384` el 24 de octubre de 2026. Esta elección reduce a la mitad las 768 características y conserva el 96.16 % de la varianza del PCA descriptivo. El F1 medio de 0.9637 queda próximo al 0.9641 del caso sin PCA, con diferencia media pareada de -0.0004; esta cercanía no demuestra equivalencia estadística. Se adopta como compromiso entre reducción de dimensionalidad y clasificación observada para la regresión logística. El detector de histogramas del punto 2 compara estos mismos casos con su propia validación interna y usa 2 componentes (Sección 3). Esta última elección constituye una variante respecto de la lectura literal del apartado 2.1, que pide reutilizar la dimensión óptima del apartado anterior. Su justificación experimental no sustituye la validación de ese alcance con el docente.

2.6. Auditoría de la cuadrícula de entrenamiento

Con las mismas 30 particiones se amplió la cuadrícula a 9 tasas (se agrega 100) y 6 checkpoints (se agrega 8000). Cada corrida seleccionó por F1 y pérdida en su propia validación interna, y volvió a entrenar con todo su entrenamiento. La cuadrícula ampliada obtuvo tasa de aciertos 0.9904 ± 0.0018 y F1 0.9636 ± 0.0068, frente a 0.9906 ± 0.0016 y 0.9641 ± 0.0063 de la vigente. La diferencia pareada de F1 fue -0.0005 ± 0.0023: mejoró en 2 corridas, empató en 22 y empeoró en 6. Aumentar la búsqueda mejoró el F1 medio de validación (0.9680 frente a 0.9673), pero no el de prueba. Se conserva la cuadrícula vigente; alcanzar sus bordes no bastaba para concluir que ampliarla mejoraría la clasificación.

En estas particiones, entre 145 y 181 mensajes de prueba por corrida tienen un texto idéntico en entrenamiento: media 162.2 de 1672 (9.70 %). Sobre los textos de prueba ausentes del entrenamiento, la regresión vigente obtiene F1 0.9565 ± 0.0076. Este diagnóstico describe otro subconjunto, con composición y dificultad propias; su diferencia con el F1 completo no mide por sí sola el sesgo por duplicados. Para evaluarlo haría falta acordar una partición agrupada por texto. El archivo original se conserva íntegro.

3. Detección de anomalías: umbral de verosimilitud

3.1. Densidades por histograma

Modelo. En cada corrida, los mensajes se proyectan con un PCA ajustado solo con el entrenamiento. Para cada componente d se estima una densidad p_d con un histograma de $B = 30$ intervalos de igual ancho entre el mínimo y el máximo observados, normalizado para que integre 1. Las componentes se tratan como independientes y el puntaje de un mensaje $\mathbf{x}$ es

$$s(\mathbf{x}) = \sum_{d=1}^{D} \log p_d(x_d). \tag{1}$$

Un valor en un intervalo vacío o fuera del soporte del histograma recibe la densidad mínima $\varepsilon = 10^{-12}$, de modo que su término vale $\log \varepsilon \approx -27.6$. Un mensaje con $s(\mathbf{x})$ menor que el umbral τ se marca como fuera de la distribución, es decir, como spam.

Decisiones del equipo. Se tomaron el 24 de octubre de 2026:

- **Datos de las densidades.** Solo los ham del entrenamiento. Los histogramas describen la distribución normal y el spam, que no participa, queda como lo atípico.
- **Umbral.** El 20 % de los ham del entrenamiento se reserva y no construye los histogramas. El umbral τ es el cuantil 5 % de los puntajes de esos ham reservados: se acepta marcar como spam alrededor del 5 % de los mensajes normales. Si τ se calculara con los mismos ham de los histogramas, ninguno caería fuera del soporte y el umbral quedaría sesgado hacia arriba frente a datos nuevos. El umbral considera la verosimilitud sumada de todas las densidades, como pide el enunciado.
- **Dimensión.** $D = 2$, elegida entre los casos del apartado 1.5 con una validación interna; se justifica a continuación.

Elección de D. El enunciado pide usar la D óptima deducida en el apartado anterior. En 1.5 se compararon 384, 192 y 2 componentes, y 384 fue el compromiso adoptado para la regresión logística (Sección 2). Ese óptimo depende del modelo, así que los mismos tres casos se compararon con el detector de histogramas en las validaciones internas. En cada una de las 30 particiones se separó la validación interna de 1.4 (20 % estratificado del entrenamiento). Con el resto se ajustaron el PCA y el detector, y este se evaluó en la validación. La Tabla 4 resume el resultado.

Alcance frente al enunciado. Usar $D = 2$ es una decisión del equipo que favorece al detector, pero difiere de reutilizar $D = 384$, la dimensión adoptada en 1.5. Queda pendiente acordar con el docente si se acepta esa variante. La auditoría adicional conserva ambos casos para hacer visible la diferencia; las métricas de $D = 2$ no sustituyen las de $D = 384$.

Alcance de la selección. Cada validación excluye la prueba de su propia corrida, pero las particiones se solapan: una fila de prueba puede participar en la validación de otra corrida. Por ello agregar las 30 validaciones para decidir una única D no aísla globalmente las 30 pruebas de esa decisión. Aquí $D = 2$ gana en todas las validaciones, de modo que elegir D separadamente en cada corrida da la misma dimensión observada. Esto no demuestra ausencia de sesgo de desarrollo ni equivalencia con datos nuevos. Las densidades se ajustan con ham; la selección de D por F1 sí usa etiquetas de spam de entrenamiento. El procedimiento completo no es enteramente no supervisado.

Tabla 4: Detector de anomalías en la validación interna de los 30 entrenamientos: media ± desviación. La última columna cuenta las particiones en que cada D obtiene el mayor F1.

D	Tasa de aciertos	F1-score	Spam detectado	Ham marcado	Gana
384	0.7812 ± 0.0141	0.0401 ± 0.0193	3.4 %	10.3 %	0
192	0.8425 ± 0.0072	0.0237 ± 0.0191	1.4 %	2.9 %	0
2	0.9201 ± 0.0106	0.7085 ± 0.0318	72.1 %	4.9 %	30

Con 384 o 192 componentes el detector casi no encuentra spam; con 2 obtiene el mayor F1 en las 30 particiones. La Figura 10 explica la diferencia. Para cada componente se calcula la media de $\log p_d$ de los ham menos la de los spam de la validación: un valor positivo indica que el spam cae en zonas de menor densidad. En la componente 1 esa brecha es 12.8: es la dirección que separa las clases. En cambio, en 330 de las 384 componentes es negativa, porque el spam está más concentrado que el ham, cerca de la moda de cada histograma, y recibe más densidad. La suma acumulada, que es la diferencia media de puntajes de un detector con k componentes, vale $+12.6$ con 2, se vuelve negativa desde 169, vale -2.8 con 192 y -41.1 con 384. Al sumar cientos de componentes poco informativas, sus aportes pequeños pero sistemáticos superan la señal de las primeras y el spam termina pareciendo más normal que el ham. La suma alcanza su máximo, 14.2, con 11 componentes; ese valor no se consideró porque el equipo restringió sus candidatos a los tres casos ensayados en 1.5. El enunciado no prescribe esa cuadrícula de búsqueda para el detector: pide reutilizar la dimensión óptima anterior.

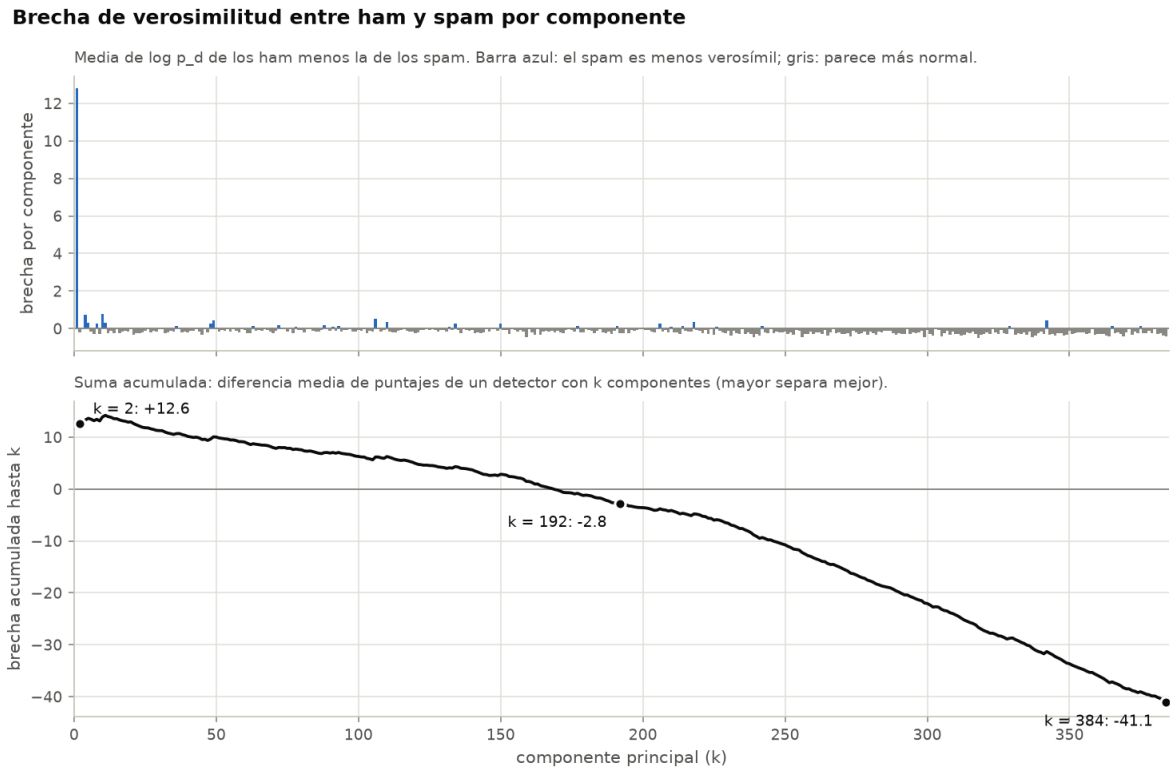


Figura 10: Brecha de verosimilitud entre ham y spam en la validación interna de la primera partición, con 384 componentes. Arriba, por componente; abajo, acumulada hasta k.

Por eso el detector usa $D = 2$ (`ANOMALY_PCA_DIMENSIONS`), mientras que la regresión conserva su óptimo de 384. Dos componentes retienen solo el 8.08 % de la varianza, pero un detector por verosimilitud no necesita reconstruir los datos: necesita las direcciones donde el spam se sale de lo normal, y aquí casi toda esa señal está en la primera componente. La regresión logística puede asignar pesos pequeños a las componentes poco útiles; el detector, en cambio, suma con el mismo peso todas las que recibe.

Densidades estimadas. La Figura 11 muestra los histogramas de la partición de la semilla 2026. De sus 3379 ham de entrenamiento, 2703 construyen los histogramas y 676 fijan el umbral, que queda en $\tau = -0.8334$. Ambas componentes son unimodales, con la mayor parte de la masa entre -0.2 y 0.3 y colas hacia valores negativos; no son exactamente simétricas ni gaussianas. Con unos 2700 ham por componente, los 30 intervalos describen la forma observada, aunque las colas pueden tener intervalos escasos o vacíos; el tamaño total no garantiza estabilidad en cada intervalo.

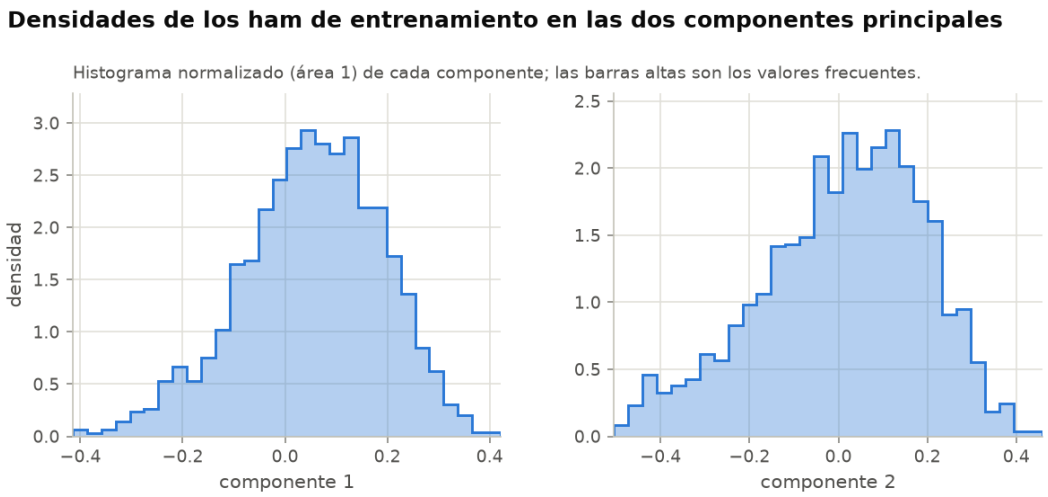


Figura 11: Densidades de los ham de entrenamiento en las dos componentes principales (semilla 2026).

Puntajes y región normal. En la prueba de esa partición (Figura 12), el 4.4 % de los ham queda bajo el umbral, cerca del 5 % buscado, y también el 75.9 % del spam. El pico cercano a −27 corresponde a mensajes fuera del soporte de algún histograma: es el 50 % del spam, a la izquierda de todos los ham en la componente 1, y solo el 0.2 % del ham. Otro 25.9 % del spam cae dentro del soporte, pero en zonas de baja densidad. La Figura 13 muestra la región que el detector acepta como normal. Como las densidades son histogramas independientes, es una unión de rectángulos alrededor del grupo de ham. El spam que se escapa (24.1 %) se mezcla con el ham en el centro del plano: en estas dos componentes no se distingue de un mensaje normal, y ningún umbral de verosimilitud lo separaría sin marcar también muchos ham. En esta partición la tasa de aciertos es 0.9294 y el F1, 0.7424; la evaluación en 30 corridas se presenta en la Sección 3.2.

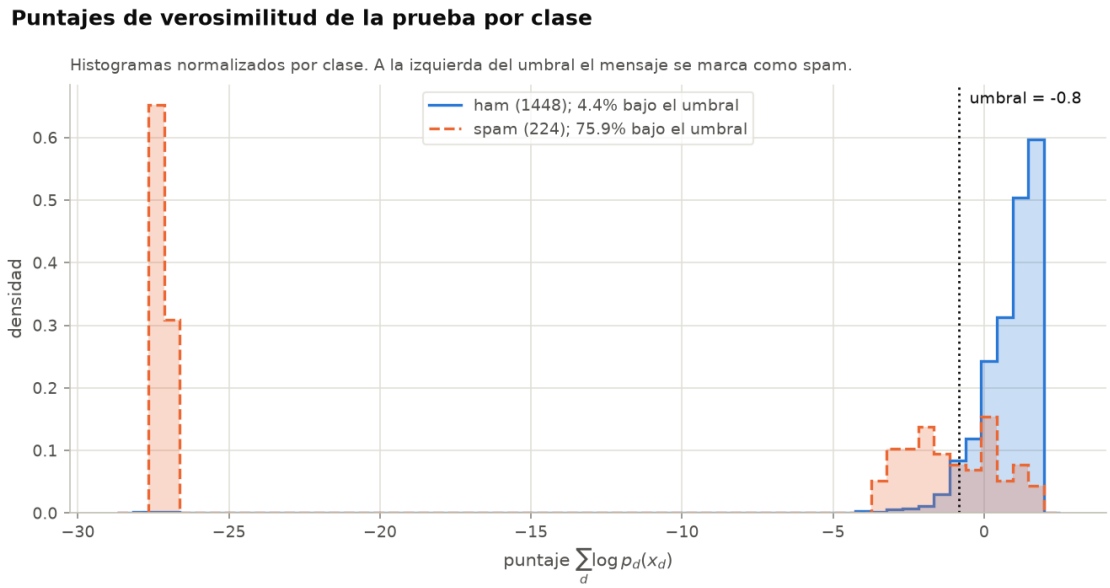


Figura 12: Puntajes de verosimilitud de la prueba por clase y umbral (semilla 2026).

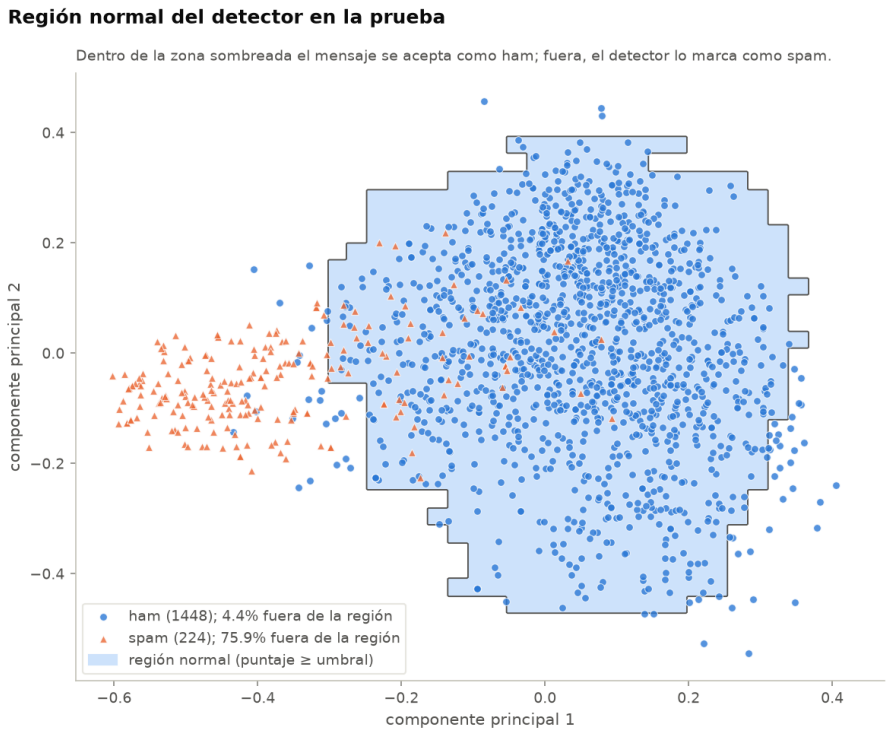


Figura 13: Región que el detector acepta como normal y mensajes de prueba (semilla 2026).

3.2. Evaluación en 30 corridas

El detector se evaluó en las mismas 30 particiones estratificadas 70/30 (semillas 2026 a 2055) que la regresión logística del punto 1. En cada una, el PCA, los histogramas y el umbral se ajustan solo con el entrenamiento y el detector se aplica una vez a la prueba. La Tabla 5 lo compara con la regresión completa y con la regresión que usa las mismas 2 componentes.

Tabla 5: Detector de anomalías y regresión logística en las mismas 30 particiones: media ± desviación de las métricas y errores promedio por prueba (1448 ham y 224 spam).

Método	Tasa de aciertos	F1-score	Ham marcado	Spam escapado	Spam detectado
Regresión, 768 dim.	0.9906 ± 0.0016	0.9641 ± 0.0063	4.0	11.8	94.7 %
Regresión, PCA $k = 2$	0.9466 ± 0.0054	0.7845 ± 0.0189	27.9	61.4	72.6 %
Histogramas, $D = 2$	0.9195 ± 0.0084	0.7043 ± 0.0247	70.5	64.0	71.4 %

En cada partición, el F1 del detector es menor que el de la regresión sin PCA, con una diferencia media de -0.2597 ± 0.0253, y menor que el de la regresión con 2 componentes, con -0.0802 ± 0.0212; en ambos casos ocurre en las 30 corridas. La tasa de aciertos baja 0.0710 y 0.0271, respectivamente. La Figura 14 muestra la distribución del F1 de todos los métodos.

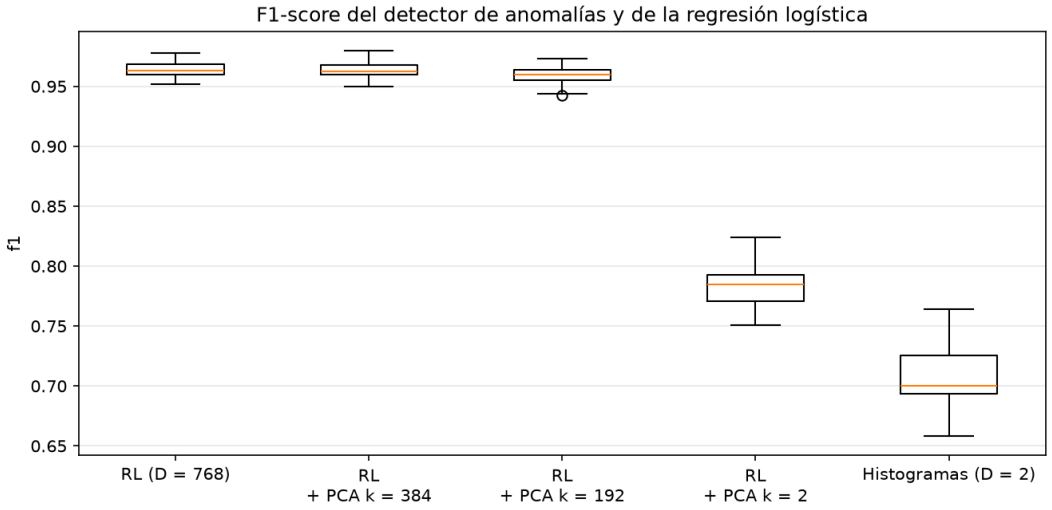


Figura 14: F1 del detector de anomalías y de la regresión logística en las mismas 30 particiones.

Validación y prueba. El F1 en la validación interna con la que se eligió D fue 0.7085 ± 0.0318 y en la prueba es 0.7043 ± 0.0247. La cercanía es descriptiva y no prueba que la selección no sobreestime el desempeño.

Con la misma información, la diferencia está en el ham. Frente a la regresión con 2 componentes, el detector encuentra casi la misma fracción de spam (71.4 % contra 72.6 %). La figura muestra spam mezclado con ham en el plano (Figura 13); los conteos agregados no permiten afirmar que ambos métodos fallen sobre los mismos mensajes. La diferencia son los falsos positivos: 70.5 ham marcados por prueba (4.9 %) contra 27.9 (1.9 %). El umbral apunta al 5 % de los ham de calibración, y el detector, que nunca ve spam, marca los ham poco frecuentes en cualquier dirección. La regresión conoce dónde está el spam y solo marca hacia ese lado. Por eso la precisión del detector es de 0.70.

La dimensión explica la mayor parte de la brecha. Reducir de 768 a 2 componentes le cuesta a la regresión 0.18 de F1; cambiar la regresión por el detector en esas 2 componentes cuesta 0.08 más. El detector no puede aprovechar más componentes porque, al sumar cientos de densidades, el spam parece más normal que el ham (Sección 3).

Tasa de aciertos y variabilidad. Con 86.6 % de ham, clasificar todo como ham ya da una tasa de aciertos de 0.866, así que el 0.92 del detector es una mejora modesta; el F1 refleja mejor el costo en la clase spam (Figura 15). El F1 del detector varía entre 0.658 y 0.764, con una desviación cuatro veces mayor que la de la regresión completa. El umbral cambia entre -1.03 y -0.49 según la partición, pero la fracción de ham marcados se mantiene en 4.9 ± 1.0 %, cerca del 5 % fijado.

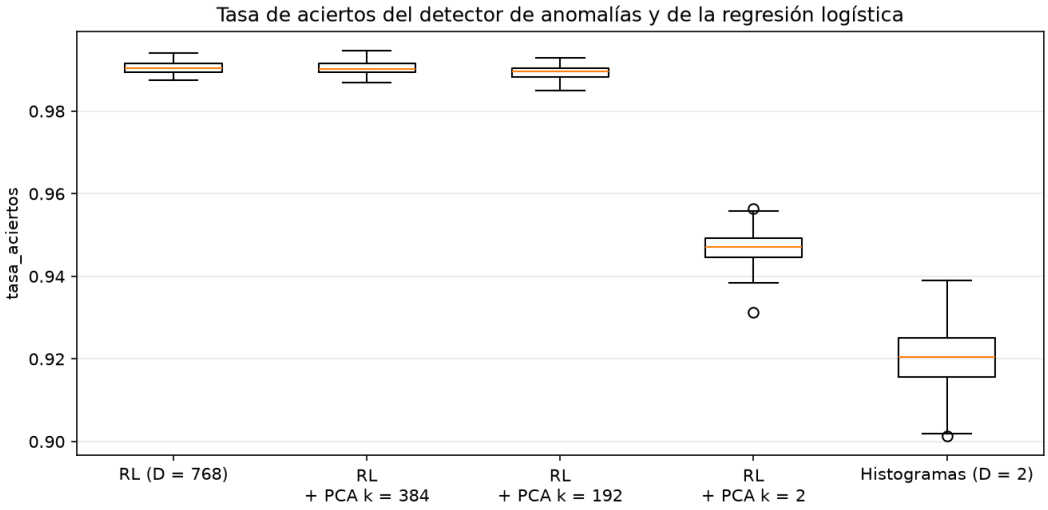


Figura 15: Tasa de aciertos del detector de anomalías y de la regresión logística en las mismas 30 particiones.

Conclusión. El detector no supera a la regresión logística en ninguna partición. Su ventaja es otra: se entrena sin ejemplos de spam y permite fijar una fracción objetivo de ham marcado en la calibración, lo que sería útil si no hubiera spam etiquetado. Con etiquetas disponibles, como en este dataset, el modelo supervisado es claramente mejor. El cuantil empírico no garantiza el mismo porcentaje en datos futuros; los empates, el tamaño de la reserva y los cambios de distribución lo afectan.

3.3. Auditoría de dimensión, intervalos y cuantil

Se ejecutaron otras 30 corridas por variante, con las mismas semillas. La referencia $D = 384$ usa los 30 intervalos y el cuantil 5 % vigentes. La alternativa calibrada elige, dentro del entrenamiento de cada corrida, $D \in \{384, 192, 2\}$, intervalos $\{15, 30, 60\}$ y cuantiles $\{0.01, 0.025, 0.05, 0.1\}$. Se maximiza el F1 de validación; los empates se resuelven por mayor tasa de aciertos y después menor dimensión, número de intervalos y cuantil. El PCA interno excluye la validación; el PCA final y las densidades excluyen la prueba. La Tabla 6 presenta medias y desviaciones entre las 30 particiones.

Tabla 6: Referencias y alternativa del detector. Los errores son promedios por prueba; las constantes oficiales permanecen sin cambios.

Variante	Tasa de aciertos	F1	Ham marcado	Spam escapado
$D = 384$, referencia	0.7892 ± 0.0119	0.0386 ± 0.0145	135.5	216.9
$D = 2$, vigente	0.9195 ± 0.0084	0.7043 ± 0.0247	70.5	64.0
Calibrada por corrida	0.9321 ± 0.0061	0.7173 ± 0.0441	35.4	78.1

La alternativa elige $D = 2$ en las 30 corridas. Elige cuantil 1 % o 2.5 % en 27 y 5 % en 3; no conserva siempre la fracción objetivo vigente. Su F1 mejora en 24 corridas y empeora en 6, con diferencia pareada $+0.0130 \pm 0.0519$. Reduce aproximadamente a la mitad los falsos positivos, pero deja escapar 14.1 spam más por prueba y aumenta la dispersión de F1. El F1 medio de validación es 0.7480 frente a 0.7173 en prueba: ampliar la selección también puede sobreajustar una validación

finita. No se adopta automáticamente: el equipo debe decidir si acepta ese intercambio y el uso de etiquetas de validación para elegir intervalos y cuantil.

La referencia $D = 384$ hace explícito el resultado de reutilizar la dimensión de 1.5, aun cuando sea desfavorable. Las alternativas se investigaron después de conocer resultados previos de este dataset: son evidencia exploratoria, no una validación independiente ni una garantía de mejora en mensajes nuevos.

4. Clasificación supervisada: modelos grandes de lenguaje

4.1. Clasificación sin ajuste

Modelo y prompt. Para el punto 3.1 se usa `Qwen/Qwen2.5-3B-Instruct`, un modelo decodificador de unos 3000 millones de parámetros ajustado para seguir instrucciones. Cada mensaje se inserta *sin preprocesar* en un prompt sin ejemplos, en inglés:

```
Classify the following SMS message as 'spam' or 'ham'. Answer with a single
word.
Message: {message}
Answer:
```

El prompt se envía como turno del usuario con la plantilla de chat del modelo. No incluye ejemplos del dataset, de modo que ningún mensaje evaluado aparece en las instrucciones.

Generación e interpretación. La generación es voraz (sin muestreo), con un máximo de 8 tokens nuevos, en GPU y con precisión de 16 bits. La respuesta se traduce a una etiqueta con la primera palabra `spam` o `ham` que contiene; una negación como *not spam* cuenta como ham. Si la respuesta no contiene ninguna de las dos palabras, se considera ham: así, un spam sin respuesta clara cuenta como spam no detectado. Se informa cuántas respuestas fueron ambiguas.

Evaluación. Con el modelo y la decodificación fijos, la predicción de un mensaje no depende de la muestra en que aparece. Por eso cada uno de los 5574 mensajes se clasifica una sola vez y las 30 muestras estratificadas del 30 % (semillas 2026 a 2055) reutilizan esas predicciones. Así las treinta evaluaciones usan exactamente las mismas predicciones para cada mensaje compartido. El modelo no se ajusta con las etiquetas SMS. Sin embargo, las muestras se toman de todo el dataset y se solapan entre sí, así que la variación entre corridas refleja solo el muestreo. Los puntos 1 y 2 se evalúan, en cambio, sobre el 30 % de prueba de cada partición.

Ejecución reproducible. Se usó la revisión `aa8e72537993ba99e69dfaafa59ed015b17504d1`, fijada tanto para el tokenizador como para los pesos. La clasificación de los 5574 mensajes tardó 84.2 segundos en una NVIDIA GeForce RTX 3080 Ti de 12 GiB, bajo WSL2, PyTorch `2.14.0+cu130`, precisión `bfloat16` y lotes de 16. Este tiempo corresponde a la generación; excluye descarga y carga del modelo. Los pesos ya estaban en caché. La generación se repitió con HF autenticado y reprodujo exactamente las respuestas de la caché anterior. El token no forma parte del notebook ni de los resultados. Después se liberó el modelo de la GPU.

Resultados. En las 30 muestras, la tasa de aciertos fue 0.7853 ± 0.0073 y el F1-score 0.5452 ± 0.0090 (media y desviación estándar muestral). No hubo respuestas ambiguas. La tabla conserva los resultados anteriores como referencia descriptiva: los protocolos de muestreo son diferentes, por lo que no es una comparación pareada sobre las mismas filas.

Tabla 7: Referencia de los puntos 1 y 2 frente a Qwen sin ajuste. Media y desviación en 30 corridas; los LLM usan muestras del dataset completo.

Método	Tasa de aciertos	F1-score
Regresión sin PCA	0.9906 ± 0.0016	0.9641 ± 0.0063
Histogramas, $D = 2$	0.9195 ± 0.0084	0.7043 ± 0.0247
Qwen2.5-3B-Instruct	0.7853 ± 0.0073	0.5452 ± 0.0090

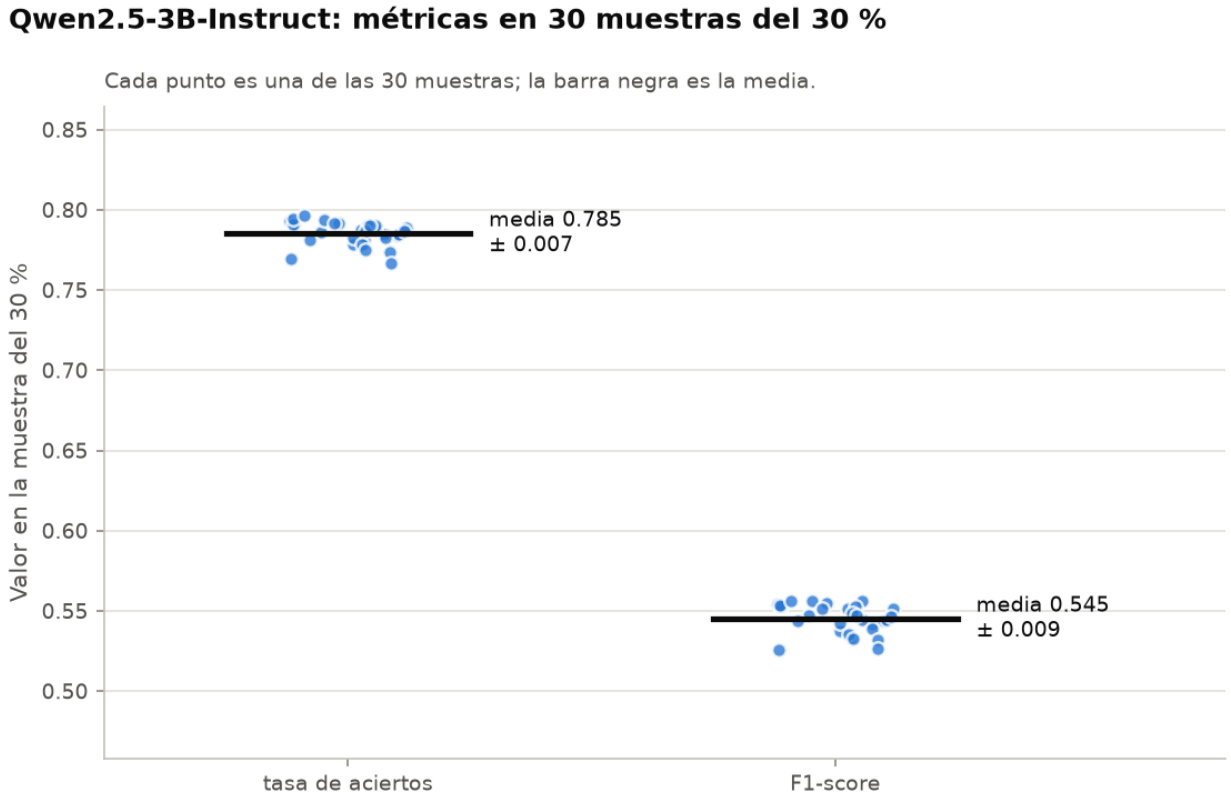


Figura 16: Variación entre muestras de las predicciones fijas de Qwen; no representa variación entre entrenamientos del modelo.

Errores y alcance. Sobre todo el dataset, Qwen detectó 717 de 747 spam (recobrado 96.0 %), pero marcó como spam 1168 de los 4827 ham (24.2 %). La precisión de spam fue 38.0 %: predijo 1885 spam, de los cuales solo 717 eran correctos. En las muestras hubo, en promedio, 349.9 falsos positivos y 9.0 falsos negativos. El F1 bajo se explica principalmente por las falsas alarmas, no por respuestas ambiguas. Incluso la tasa de aciertos queda por debajo de predecir siempre ham (86.6 % en el dataset), aunque esa regla trivial tendría F1 cero para spam.

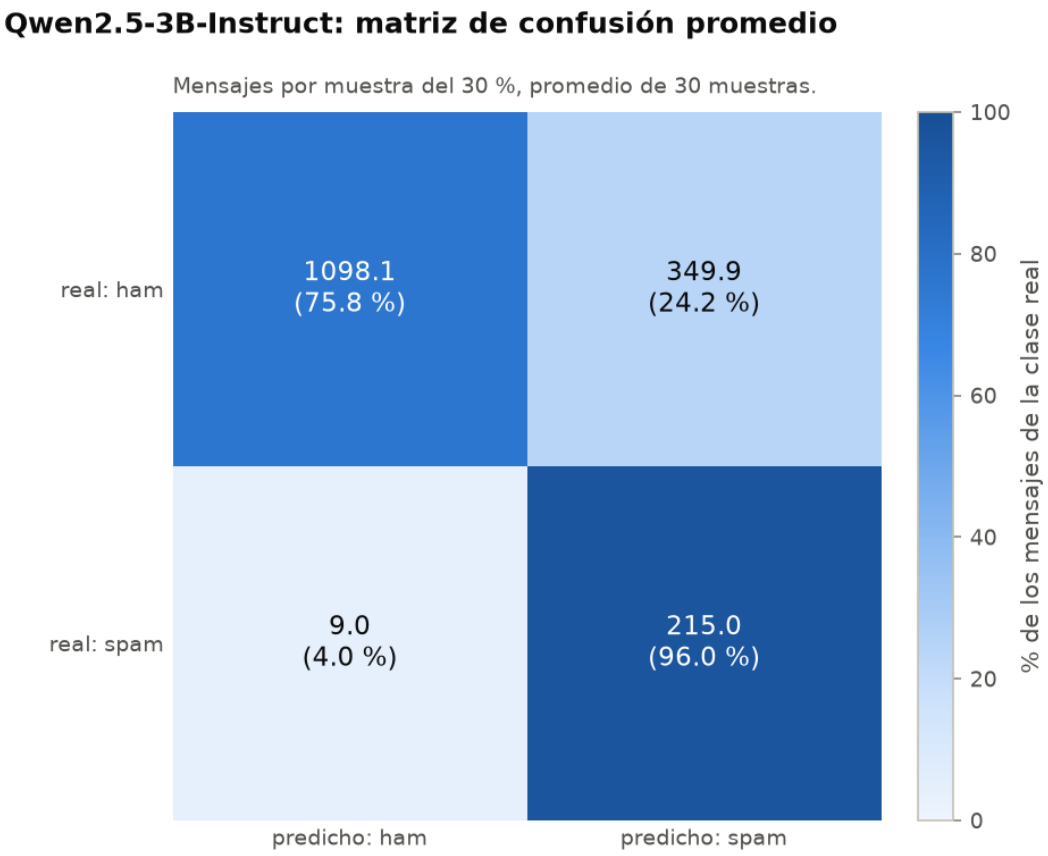


Figura 17: Confusión promedio de Qwen. La abundancia de ham marcado como spam limita la precisión a pesar de detectar la mayor parte del spam.

Este resultado corresponde al modelo, prompt sin ejemplos y decodificación confirmados. No se cambió el prompt después de observar las métricas. Tampoco permite concluir que cualquier LLM sea inferior: evalúa esta configuración concreta. El apartado siguiente conserva esta línea base y agrega los dos modelos elegidos para 3.2.

4.2. Modelos adicionales con cuantización NF4

Selección y protocolo. Después de respaldar 3.1, el equipo confirmó Mistral-7B-Instruct-v0.3 [8] y Qwen2.5-14B-Instruct [10], con aproximadamente 7.25 y 14.77 mil millones de parámetros. Ambos se cargan a 4 bits mediante NF4, con doble cuantización y cálculo bfloat16 [6]. Se cuantizan las capas lineales al cargar el checkpoint original; otras matrices pueden conservar mayor precisión. No se entrenan adaptadores ni se modifica el modelo con las etiquetas SMS.

Se mantienen textos originales, prompt sin ejemplos, generación voraz de hasta ocho tokens, interpretación de respuestas y las mismas 30 muestras estratificadas. Cada tokenizador aplica su propia plantilla de chat. La comparación cambia familia, tamaño y precisión de pesos simultáneamente: compara configuraciones prácticas, sin atribuir causalmente toda diferencia al número de parámetros o a cuantizar. No se auditó si los corpus de preentrenamiento contienen mensajes de esta colección pública; no puede descartarse exposición previa al dataset.

Reproducción y memoria. Los modelos se ejecutan de forma secuencial en la misma RTX 3080 Ti, liberando uno antes de cargar el otro. Las revisiones fijadas son:

- Mistral 7B: `c170c708c41dac9275d15a8fff4eca08d52bab71`.
- Qwen 14B: `cf98f3b3bbb457ad9e2bb7baf9a0125b6b88caa8`.

Se usan Transformers 5.20.2, bitsandbytes 0.50.2 y Accelerate 1.15.0. La carga exige que el modelo completo resida en GPU; la prueba rápida incluye el prompt más largo del dataset, sin truncamiento. Las cachés NF4 registran revisión, cuantización, precisión, lote y versiones para no confundirlas con respuestas de precisión completa. Los tiempos de generación excluyen descarga y carga; el pico de memoria se mide durante la generación.

Resultados. Se completaron 5574 respuestas por modelo y la evaluación de las mismas 30 muestras. La tabla informa media y desviación estándar muestral; las respuestas de Qwen 3B se reutilizan como referencia.

Tabla 8: Tres configuraciones de lenguaje sobre las mismas 30 muestras del 30 %.

Configuración	Tasa de aciertos	F1-score
Qwen 3B, BF16	0.7853 ± 0.0073	0.5452 ± 0.0090
Mistral 7B, NF4	0.7923 ± 0.0084	0.5518 ± 0.0108
Qwen 14B, NF4	0.8954 ± 0.0065	0.7126 ± 0.0139

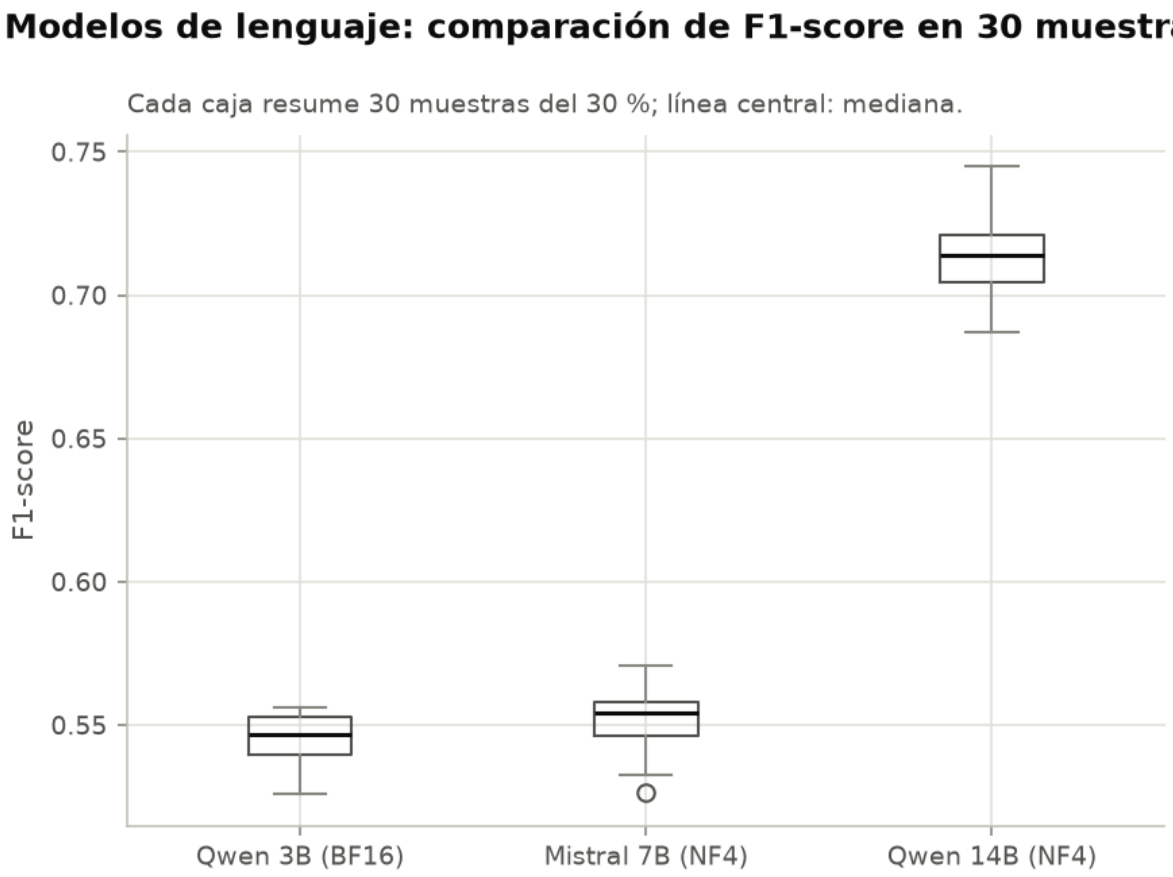


Figura 18: F1 de las predicciones fijas de cada configuración. Las cajas resumen muestras solapadas, no entrenamientos independientes.

Errores e interpretación. Qwen 14B mejora principalmente la precisión de spam: reduce los falsos positivos de 1168 a 552 frente a Qwen 3B y detecta 721 de los 747 spam (recobrado 96.5 %). Su precisión es 56.6 %; las falsas alarmas siguen limitando el F1. Mistral queda cerca de la referencia. Sus doce respuestas ambiguas corresponden a ham y no añaden spam escapado al aplicar la regla acordada.

Tabla 9: Errores sobre los 5574 mensajes; no son promedios por muestra.

Configuración	Falsos positivos	Falsos negativos	Ambiguas
Qwen 3B, BF16	1168	30	0
Mistral 7B, NF4	1122	35	12
Qwen 14B, NF4	552	26	0

Las diferencias pareadas de F1, calculadas por muestra, son +0.0066 para Mistral menos Qwen 3B (positiva en 23/30), +0.1675 para Qwen 14B menos Qwen 3B y +0.1609 para Qwen 14B menos Mistral (positivas en 30/30). Se trata de evidencia descriptiva: el solapamiento entre muestras impide tratarlas como treinta experimentos independientes. Tampoco se aísla el efecto causal del tamaño o de la cuantización. El prompt se mantuvo fijo después de observar estos resultados.

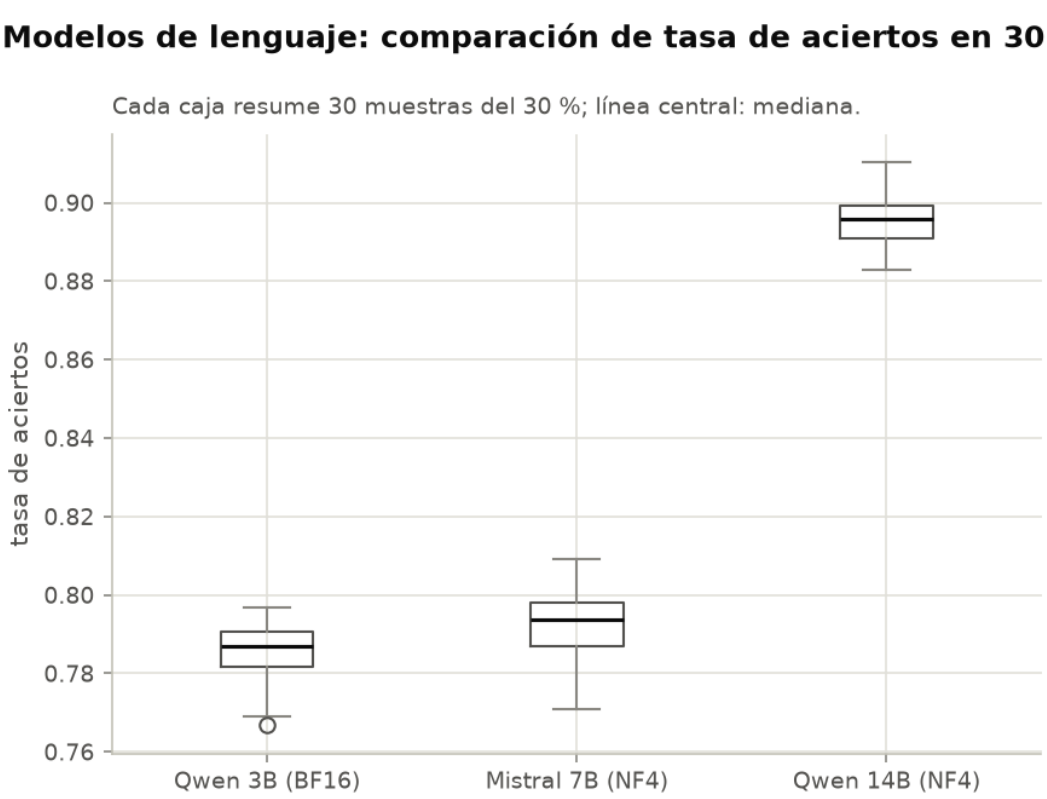


Figura 19: Tasa de aciertos bajo el mismo muestreo. Se informa junto con F1 para reflejar el desbalance entre ham y spam.

Recursos observados. Los pesos y buffers ocupan 3.75 GiB en Mistral y 9.05 GiB en Qwen 14B. La prueba rápida incluyó el prompt máximo de cada tokenizador (307 y 312 tokens), sin truncamiento. Los lotes se calificaron antes de la inferencia completa, según la memoria disponible.

Tabla 10: Generación completa en la RTX 3080 Ti. Los tiempos excluyen descarga y carga; los picos pertenecen al asignador de PyTorch.

Modelo NF4	Lote	Tiempo (s)	Asignado (GiB)	Reservado (GiB)
Mistral 7B	16	168.6	4.625	5.078
Qwen 14B	4	536.8	9.774	9.984

Estos picos no incluyen la memoria de otros procesos ni del controlador. Se liberó cada modelo al terminar. Las respuestas se transfirieron con SHA-256 y las 30 filas por modelo se reprodujeron en CPU desde las cachés, con tolerancia 10^{-12} respecto de los CSV. Las figuras individuales, las comparativas y las tablas salen de las celdas del notebook.

4.3. Ajuste fino eficiente

Decisión y alcance. Para 3.3 se eligió ajustar Qwen2.5-14B-Instruct, uno de los dos modelos adicionales de 3.2, mediante LoRA [4] sobre su base NF4 congelada. Esta combinación sigue el enfoque de ajuste sobre pesos cuantizados de QLoRA [3, 5]; no se utiliza DoRA [7]. El equipo

confirmó PEFT para los adaptadores y un bucle explícito de entrenamiento en PyTorch. El enunciado de 3.3 pide entrenar y analizar, sin fijar 30 entrenamientos; se acordó un único piloto.

Adaptación de bajo rango. Para cada matriz lineal congelada $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, se aprende una actualización

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

El rango es $r = 8$ y la escala $\alpha = 16$. Solo se optimizan las matrices de adaptación: 34 406 400 parámetros. Se adaptan las proyecciones de atención `q/k/v/o_proj` y las del bloque de avance `gate/up/down_proj`, con dropout 0.05 y sin entrenar sesgos. La base conserva NF4 y sus matrices no cuantizadas BF16. Las matrices LoRA y los estados del optimizador usan FP32; el cálculo CUDA usa BF16.

Datos y separación. Se fija la semilla 2026 para una partición estratificada externa 70/30. De sus 3902 filas de entrenamiento se reserva 20 % para validar, mediante otra partición estratificada con semilla 2027. Por tanto, el ajuste recibe 3121 mensajes (2703 ham y 418 spam), la validación 781 (676/105) y la prueba 1672 (1448/224). Las filas de prueba no intervienen en el gradiente ni en la elección de parámetros.

La separación es por filas y conserva los duplicados del archivo UCI. Hay 127 textos de prueba presentes en el ajuste efectivo y 150 en el entrenamiento externo completo. Esto limita la interpretación como generalización a textos nunca vistos. Se informa también un diagnóstico sobre las 1545 filas de prueba sin texto presente en ajuste; usa los mismos modelos y predicciones, sin modificar la partición principal ni reentrenar.

Objetivo supervisado. Se mantiene exactamente el prompt y la plantilla de chat de 3.2. Cada ejemplo agrega la respuesta correcta `ham` o `spam` y el token de fin de respuesta. La entropía cruzada causal solo supervisa esos tokens; prompt y padding se enmascaran con -100. Para un conjunto de respuestas R_i, la pérdida media es

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \frac{1}{|R_i|} \sum_{t \in R_i} \log p_\theta(y_{i,t} \mid \text{prompt}_i, y_{i,<t}).$$

La cabeza calcula logits únicamente en las posiciones necesarias para esa pérdida, conservando el contexto causal completo. Una prueba compara su valor y sus gradientes con el cálculo completo enmascarado. El padding se enmascara por posición, conservando el EOS real aunque comparta su identificador con el token de relleno.

Optimización y memoria. Se confirmó una época de AdamW con tasa constante 2×10^{-4}, decaimiento de pesos cero, recorte de norma de gradiente a 1, lote físico de un mensaje y acumulación de 16. El último grupo parcial se normaliza por su tamaño real. Se usa gradient checkpointing no reentrante para recalcular activaciones en backward y reducir memoria. Se guardan adaptadores y estados de progreso cada 50 pasos, fuera de Git.

La prueba de capacidad ejecutó forward, backward, acumulación y dos pasos de AdamW con el ejemplo de ajuste más largo (314 tokens), alcanzando 10.1925 GiB asignados y 10.3359 GiB reservados. Después restauró las matrices LoRA y el estado aleatorio antes del ajuste definitivo. Los pesos y buffers del modelo preparado ocupan 9.18 GiB. La revisión del modelo base y las versiones de 3.2 se conservan; se añadió solo PEFT 0.21.0.

Evaluación. Se comparan las respuestas de la base de 3.2 y del modelo ajustado sobre las mismas 1672 filas reservadas. Se conservan generación voraz, hasta ocho tokens y la misma interpretación de respuestas. Las métricas son tasa de aciertos y F1 con spam positivo. Al tratarse de una sola partición, no se presentan desviaciones entre 30 entrenamientos.

Entrenamiento. La época completa tuvo 196 pasos de AdamW y tardó 3498.5 s en la RTX 3080 Ti, incluyendo la validación final y excluyendo carga, capacidad y validación inicial. Los picos fueron 10.19 GiB asignados y 10.37 GiB reservados. La pérdida de validación sobre las 781 filas reservadas bajó de 0.5163 antes del ajuste a 0.0446 después de la época (Figura 20). La pérdida por paso cae en los primeros diez pasos y luego queda cerca de cero, con picos aislados (máximo 0.39 después del paso 10). Dos puntos de validación no bastan para describir una curva de sobreajuste.

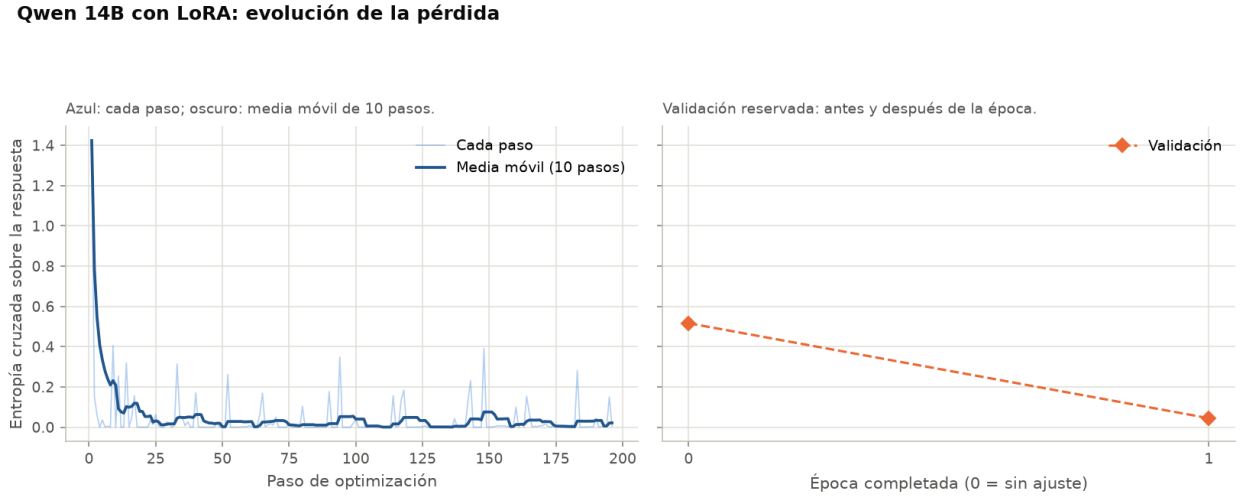


Figura 20: Pérdida de entrenamiento por paso y de validación antes y después de la única época. La validación usa solo filas del entrenamiento externo.

Resultados. La inferencia de las 1672 filas de prueba tardó 224.2 s con lote 4 (pico de 9.81 GiB asignados) y produjo solo respuestas `ham` o `spam`, sin ambiguas. Después se liberó el modelo. La Tabla 11 compara la base de 3.2 y el modelo ajustado sobre las mismas filas. Las métricas se recalcularon en CPU desde las cachés transferidas con SHA-256 y coinciden con las tablas exportadas.

Tabla 11: Qwen2.5-14B-Instruct NF4 antes y después de LoRA, en la prueba fija de 1672 filas (1448 ham y 224 spam). Un entrenamiento y una partición.

Modelo	Tasa de aciertos	F1	VP	FP	FN	VN
Sin ajuste	0.8935	0.7120	220	174	4	1274
Con LoRA	0.9922	0.9708	216	5	8	1443

El ajuste ataca el problema principal de 3.1 y 3.2: los falsos positivos bajan de 174 a 5 (del 12.0 % al 0.3 % de los ham) y la precisión para spam sube de 55.8 % a 97.7 %. A cambio, los falsos negativos pasan de 4 a 8, y el recobrado de spam baja de 98.2 % a 96.4 %. En las filas emparejadas, LoRA corrige 172 errores de la base e introduce 7 nuevos (Figura 21).

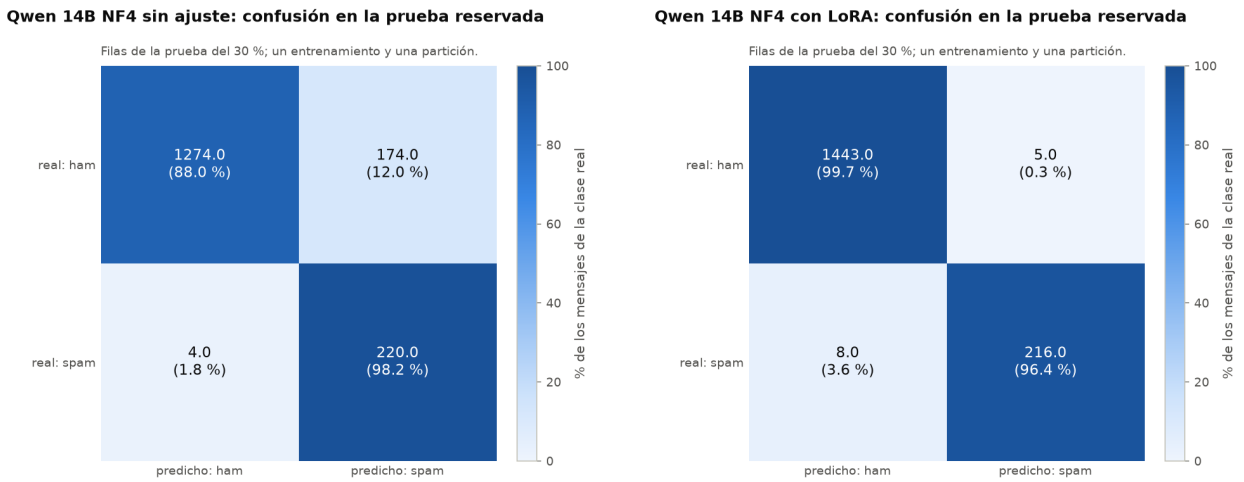


Figura 21: Matrices de confusión de Qwen 14B NF4 sin ajuste (izquierda) y con LoRA (derecha) en la misma prueba reservada.

Limitaciones. Hay un solo entrenamiento, así que no se mide la variación entre entrenamientos ni entre particiones. Los hiperparámetros se fijaron antes de entrenar, y la prueba no se usó para elegirlos ni para elegir checkpoints. En las 1545 filas de prueba sin texto presente en el ajuste, la mejora se mantiene: F1 de 0.7024 a 0.9688 y tasa de aciertos de 0.8958 a 0.9922. Solo 1 de los 13 errores del modelo ajustado corresponde a una fila repetida. La regresión logística del punto 1 promedia F1 0.9641 ± 0.0063 en 30 particiones; la cifra de 3.3 sale de una partición, así que la comparación es descriptiva. Además, el ajuste exige GPU, casi una hora de entrenamiento y 34 406 400 parámetros LoRA, mientras que la regresión trabaja sobre características ya calculadas.

5. Pruebas unitarias

El enunciado exige al menos dos pruebas unitarias para `forward` y dos para `train_logistic_regression`. Las pruebas están en `tests/test_punto1.py` y se ejecutan en la sección "Pruebas unitarias" del notebook y con `pytest`. Cada prueba es una función documentada con Doxygen que falla mediante `assert` o `torch.testing.assert_close` si el resultado no coincide con el esperado.

La implementación que se prueba vive en el mismo notebook. El cargador `tests/notebook_loader.py` ejecuta únicamente las celdas cuyo prefijo es `# Definiciones:`; excluye la configuración, la descarga de modelos y las corridas experimentales. Así las pruebas verifican las funciones entregadas en el cuaderno sin mantener una copia separada en archivos por punto. Las pruebas de estructura comprueban que el notebook sea válido, que no importe el antiguo paquete y que las definiciones no se dupliquen. La última ejecución local de `pytest` (26/10/2026) dio 77 pruebas superadas, ninguna pendiente y ninguna fallida.

El ejecutor del notebook usa pytest para resolver fixtures como `monkeypatch` y `tmp_path`; llamar las pruebas sin argumentos producía fallos en las pruebas nuevas de 3.1. Una prueba de regresión verifica la ejecución con fixtures. La celda recarga el cargador y los módulos de pruebas para no usar definiciones antiguas en un kernel persistente. También se verifica que tokenizador y pesos compartan la misma revisión, aunque Transformers no conserve `config._commit_hash`. Las dos verificaciones usan simulaciones o archivos temporales, sin descargar modelos. La celda del notebook ejecutada en WSL dio 64/0/0 al cerrar 3.2.

Las trece pruebas adicionales de `tests/test_punto3_finetuning.py` comprueban el objetivo causal enmascarado, la conservación de EOS real cuando comparte identificador con padding, el rechazo de etiquetas o longitudes incompatibles, la equivalencia de pérdida y gradientes con el cálculo completo, la restauración después de la prueba de capacidad, la acumulación en el último grupo parcial, la separación de filas de ajuste/validación/prueba, la base BF16 congelada, el contrato de clasificación y el título exportable de la figura. También rechazan respuestas e historial procedentes de adaptadores distintos. Usan modelos diminutos o simulaciones, sin descargar pesos. La celda de pruebas del kernel WSL, con el notebook y las pruebas sincronizados, también dio 77/0/0 (26/10/2026).

5.1. Evaluación del modelo

`test_forward_zero_parameters_returns_one_half`

Evalúa cinco muestras aleatorias en $\mathbb{R}^3$ con $w = 0$ y $b = 0$. El logit es 0 para cualquier entrada y $\sigma(0) = 0.5$, así que todas las probabilidades deben valer 0.5. La prueba verifica además que la salida tenga forma $(5,)$, una probabilidad por muestra.

`test_forward_matches_sigmoid_and_is_stable`

Con $X = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ -1 & -1 \end{pmatrix}$, $w = (0.5, -1)$ y $b = 0.25$, los logits son -1.25, 0.25 y 0.75; las probabilidades se comparan con $1/(1 + e^{-z})$ calculado con la biblioteca estándar de Python. Luego evalúa logits de ± 1000: la salida debe valer 1 y 0 y el gradiente respecto de w debe ser finito. Verifica la fórmula y la estabilidad numérica; la forma directa de la sigmoide no supera esta prueba porque su gradiente resulta indefinido.

Una tercera prueba de `forward`, `test_forward_gradient_at_zero_matches_sigmoid_derivative`, comprueba $\partial p/\partial w = x/4$ y $\partial p/\partial b = 1/4$ con logit cero. Reprodujo una derivada nula en la versión anterior y pasa con la selección explícita de la rama del exponente.

5.2. Entrenamiento

`test_train_follows_cross_entropy_gradient`

Sobre 200 puntos en $\mathbb{R}^3$ con clases que se traslapan, compara el primer paso del entrenamiento con $w_1 = w_0 - \alpha \nabla_w L(w_0)$ y $b_1 = b_0 - \alpha\, \partial L/\partial b$, donde el gradiente de referencia se obtiene con Autograd sobre `binary_cross_entropy_with_logits` de PyTorch, independiente de la fórmula cerrada que usa la implementación. También comprueba que la pérdida inicial registrada coincida con la de referencia y que, en 50 épocas, la pérdida no aumente en ninguna época y termine por debajo de la inicial. Una implementación que no promedie el gradiente entre las muestras no supera esta prueba.

`test_train_separates_linearly_separable_data`

Entrena sobre seis puntos en dos grupos separables por una recta y verifica que el modelo clasifique correctamente todos los puntos. Entrena una segunda vez con la misma semilla y verifica que los pesos y el sesgo sean exactamente iguales, es decir, que el entrenamiento sea reproducible.

5.3. Otras pruebas del punto 1

Además de las cuatro pruebas obligatorias, `tests/test_punto1.py` verifica el resto del punto 1 sin descargar datos ni modelos:

`test_make_linearly_separable_shapes_and_reproducibility`
: El generador de 1.3 devuelve las formas y tipos del contrato, las dos clases balanceadas, todos los puntos a una distancia de al menos el margen de la frontera verdadera y del lado de su clase, y los mismos datos con la misma semilla.

`test_extract_features_keeps_order_and_contract`
: Con un codificador falso, `extract_features` devuelve un tensor `float32` de forma (n, D) en el orden de los textos.

`test_grid_search_covers_grid_and_orders_by_score`
: La calibración prueba todas las combinaciones, deja primero la de mayor puntaje y resuelve los empates con la menor pérdida en validación.

`test_boundary_plots_return_figures`, `test_run_plots_return_figures`
: Las siete gráficas de 1.3 y 1.4 devuelven una figura con el título pedido, sin mostrarla.

`test_fit_pca_recovers_dominant_direction`
: Con 500 puntos en $\mathbb{R}^3$ alargados sobre la dirección $(0.6, 0.8, 0)$ y desplazados del origen, la primera componente debe coincidir con esa dirección (salvo el signo) y explicar más del 99 % de la varianza; la media debe ser la de los datos y las componentes, ortonormales y ordenadas de mayor a menor varianza.

`test_transform_pca_matches_explained_variance`
: Los datos de ajuste proyectados deben quedar centrados y sin correlación, con la varianza explicada de cada componente en la diagonal de su covarianza; las fracciones de varianza deben sumar 1 con todas las componentes; la media del ajuste se debe proyectar en el origen, y ajustar dos veces debe dar exactamente las mismas componentes.

`test_pca_plots_return_figures`
: Las gráficas de varianza explicada y de proyección (con 2 y con más componentes) devuelven una figura con el título pedido y la cantidad de paneles esperada.

5.4. Protocolo experimental y caché

Las cuatro pruebas de `tests/test_experiment_protocol.py` usan tensores pequeños y reproducibles, sin descargar modelos ni datos:

1. Comparan todos los candidatos de una cuadrícula vectorizada con entrenamientos independientes, incluyendo el estado inicial y varios checkpoints. Verifican F1 y pérdidas de ajuste y validación.

2. En dos semillas comprueban que el ajuste y la validación interna pertenecen al entrenamiento de esa corrida, que son disjuntos y que excluyen la prueba. También verifican que el entrenamiento final use la tasa y las épocas del candidato ganador.

3. Comprueban que el PCA interno se ajuste sin validación ni prueba, que el PCA final use solo el entrenamiento completo y que distintos valores de k reutilicen únicamente ajustes con los mismos índices.

4. Rechazan cachés antiguas o incompatibles por orden de los textos, identificador o revisión del modelo, forma, tipo o valores no finitos.

5.5. Detector de anomalías (punto 2.1)

Las quince pruebas de `tests/test_punto2.py` usan tensores pequeños construidos a mano:

- **Histogramas.** Cada densidad integra 1; los bordes internos pertenecen al intervalo de la derecha y el máximo se cuenta en el último; una columna constante recibe un soporte de ancho positivo y puntajes finitos.
- **Verosimilitud.** El puntaje es la suma de los logaritmos de las densidades marginales y coincide con la suma de los términos por dimensión; los intervalos vacíos y los valores fuera del soporte, por ambos lados, usan la densidad mínima sin recortar los valores.
- **Umbral.** El cuantil se interpola linealmente; con empates, la regla estricta puede marcar menos de la fracción pedida. El detector construye los histogramas y fija el umbral con filas normales disjuntas, y es reproducible con la misma semilla.
- **Selección y diagnóstico.** Cada candidato usa sus primeras k columnas; cambiar el spam de ajuste no cambia el detector, porque solo los ham construyen las densidades. La brecha por componente es positiva donde el spam cae en zonas de baja densidad.
- **Validaciones y figuras.** Se rechazan datos vacíos, mal formados o no finitos, cantidades de intervalos y cuantiles inválidos; las cuatro figuras devuelven su título como `suptitle`.

5.6. Ejecución portátil de los puntos 1 y 2

Tres pruebas adicionales comprueban que el ejecutor compartido seleccione los puntos 1 y 2 y las pruebas sin modificar la configuración original, que valide la caché sin tocar resultados existentes y que ejecute una celda mínima en un kernel limpio del mismo intérprete invocante. No descargan modelos ni ejecutan experimentos largos. El procedimiento para otras computadoras se describe en `docs/reproducibility.md`.

5.7. Modelos de lenguaje (puntos 3.1 y 3.2)

Las quince pruebas de `tests/test_punto3.py` no descargan ni ejecutan modelos: usan un tokenizador y un modelo simulados que imitan la interfaz de Hugging Face.

- **Interpretación.** `parse_label` traduce respuestas como `spam`, `SPAM` con salto de línea o `Ham.`; una negación (*not spam*) cuenta como ham y una respuesta sin ninguna de las dos palabras, también, y queda marcada como ambigua.
- **GPU obligatoria.** Sin GPU CUDA, la selección del dispositivo se detiene con un error salvo que se permita la CPU.
- **Generación por lotes.** Aunque los lotes se agrupan por longitud, las respuestas vuelven en el orden original; solo se decodifican los tokens nuevos, sin muestreo, y `classify_messages` devuelve un tensor `int64` de forma $(n,)$.
- **Caché y muestras.** La caché de respuestas se rechaza si cambian los textos o el prompt; cada corrida evalúa la muestra estratificada de su semilla e incluye la matriz de confusión.

- **Cuantización NF4.** El cargador fija la revisión y el mapa de GPU sin llamar después a `to` sobre pesos de precisión completa; rechaza CPU y revisiones mutables antes de importar las dependencias.
- **Identidad del perfil.** La caché NF4 cambia al modificar revisión, lote o precisión de cálculo y no se confunde con la de precisión completa. El perfil sin cuantización conserva exactamente la identidad de la caché de 3.1.

6. Comparación de resultados

TODO: comparar todos los métodos con la tabla y los diagramas de caja de la sección "Comparación de resultados" del notebook.

7. Conclusiones

TODO: conclusiones del trabajo.

7.1. Limitaciones

TODO: limitaciones del protocolo y de los modelos.

Referencias

[1] Tiago A. Almeida and José María Gómez Hidalgo. SMS spam collection. UCI Machine Learning Repository, `https://archive.ics.uci.edu/dataset/228/sms+spam+collection`, 2011.

[2] Tiago A. Almeida, José María Gómez Hidalgo, and Akebo Yamakami. Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering*, pages 259–262, 2011.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[5] Hugging Face. PEFT: quantization and LoRA training. `https://huggingface.co/docs/peft/developer_guides/quantization`, 2026. Documentación y código de PEFT 0.21.0; consultados el 26 de octubre de 2026.

[6] Hugging Face. Transformers: Bitsandbytes quantization. `https://huggingface.co/docs/transformers/quantization/bitsandbytes`, 2026. Consulta: 26 de octubre de 2026.

[7] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353*, 2024.

[8] Mistral AI. Mistral-7B-Instruct-v0.3: Model card. `https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3`, 2024. Consulta: 26 de octubre de 2026.

[9] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[10] Qwen Team. Qwen2.5-14B-Instruct: Model card. `https://huggingface.co/Qwen/Qwen2.5-14B-Instruct`, 2024. Consulta: 26 de octubre de 2026.

[11] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[12] Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70):1–57, 2018.